

密级: 公开
编号: 22212713



硕士学位论文

基于 GPU 的 Δ -Stepping 算法研究

Research on Δ -Stepping Algorithm Based on GPU

学位申请人: 刘晶
导师姓名及职称: 胡平 副教授 姚正安 教授
专业名称: 应用统计

2024 年 5 月 21 日

中山大学硕士学位论文

基于 GPU 的 Δ -Stepping 算法研究
Research on Δ -Stepping
Algorithm Based on GPU

专业: 应用统计
学位申请人: 刘晶
指导教师: 胡平 副教授
姚正安 教授

论文答辩委员会主席: _____
成员: _____

二〇二四年五月

摘 要

单源最短路径问题是图论中的经典问题，不仅应用于路径规划领域，在其它领域比如社交网络、生物学、通信网络等都发挥重要作用。然而，随着数据量的激增和计算能力的持续发展，人们对最短路径计算的时效性要求日益提高，因此也对最短路径算法的速度提出了更高的要求。

Δ -Stepping 算法是目前较为成熟的单源最短路径算法，“桶”的引入使其兼具 Dijkstra 算法的精度及 Bellman-Ford 算法的效率，但是 Δ -Stepping 算法仍存在问题：(1) 轻重边的划分未考虑算法内外循环的作用不同，从而造成轻边松弛次数的浪费，且未利用顶点度数和边权重信息剔除无效边及识别叶顶点；(2) Δ 参数值的选取具有随机性，未考虑顶点度数及边权重等特性；(3) 现有的并行方案主要基于 MPI 框架在 CPU 上进行并行化处理，而对于 GPU 上的并行化则考虑得较少。

针对上述问题，本文主要研究内容包括：(1) **松弛次数优化**。首先对轻重边重新定义，使得轻边列表处理反复落入当前桶中的顶点，减少轻边松弛次数。其次根据图中边和点的特性，将无效边和叶顶点进行识别并处理；(2) **Δ 参数值的优化**。将最短路径树的平均距离与平均深度的商值作为 Δ 值，这样可以有效平衡长距离最短路径和短距离最短路径的探索，有利于在算法的初期阶段快速收敛；(3) **基于 GPU 的并行**。利用 CUDA 并行框架，对 Δ -Stepping 算法采取清桶阶段顶点并行与松弛阶段边并行的混合并行方式。

最后本文在常见网络图和真实图上进行实验分析，结果表明：(1) 在串行情况下，松弛算法优化效果比较好，且稠密图上的优化效果总体表现优于稀疏图，平均加速率可达 15.5%，加速率最高可达 70%；(2) 在串行情况下，最短路径树确定 Δ 值是随机给定一组 Δ 值中表现比较好的，即使不是最优值，也是次优值；(3) 在 GPU 并行处理的情况下，原始算法的整体运行时间开销仅为串行实现的 1%。并且通过引入松弛算法的优化以及 Δ 参数值的优化这两种策略，算法的加速率最高可分别达到 59% 和 51%。

关键词：单源最短路径， Δ -Stepping 算法，松弛次数优化，最短路径树，GPU 并行

ABSTRACT

The single-source shortest path problem is a classical problem in graph theory, which is not only applied in the field of path planning, but also plays an important role in other fields such as social networks, biology, and communication networks. However, with the proliferation of data volume and the continuous development of computational power, the timeliness of shortest path computation is increasingly required, and thus the speed of shortest path algorithms is required to be higher.

The Δ -Stepping algorithm is currently a more mature single-source shortest path algorithm, the introduction of “bucket” makes it both the accuracy of Dijkstra algorithm and the efficiency of Bellman-Ford algorithm, but Δ -Stepping algorithm there are still some issues: (1) The division of light and heavy edges doesn’t take into account the different roles of the inner and outer loops of the algorithm, which results in a waste of the number of light edge relaxations, and doesn’t utilize the vertex degree and edge weight information to eliminate invalid edges and identify leaf vertices; (2) The selection of the value of parameter Δ has randomness, without considering properties such as vertex degree and edge weights; (3) Existing parallel schemes are mainly based on the MPI framework for parallelization on CPUs, while less consideration has been given to parallelization on GPUs.

To address the above problems, the main research content of this paper includes: (1) **Relaxation Count Optimization.** Firstly, the redefinition of light and heavy edges reduces the number of relaxations for vertices repeatedly falling into the current bucket. Secondly, based on the characteristics of edges and vertices in the graph, invalid edges and leaf vertices are identified and processed; (2) **Optimization of the value of the Δ parameter.** The quotient of the average distance to the average depth of the shortest path tree is used as the Δ of value, effectively balancing the exploration of long-distance and short-distance shortest paths and facilitating rapid convergence in the early stages of the algorithm; (3) **GPU-based parallelism.** Based on the CUDA parallel framework, a mixed parallelization approach of vertex parallelism in the bucket clearing phase and edge parallelism in the relaxation phase is adopted for the Δ -Stepping algorithm.

Finally, this paper carries out experimental analysis on common network graphs and

real graphs, and the results show that: (1) In the serial case, the relaxation algorithm optimization effect is better, and the optimization effect on dense graphs is better than that on sparse graphs in general, with an average acceleration rate of up to 15.5%, and a maximum acceleration rate of up to 70%; (2) In the serial case, the shortest path tree determines that the value of Δ is the better performing, if not the optimal, suboptimal value of a randomly given set of the values of Δ ; (3) In the case of GPU parallel processing, the overall runtime overhead of the original algorithm is only 1% of the serial implementation. And by introducing the optimization of the relaxation algorithm and the optimization of the value of the Δ parameter, the algorithm's speedup rate can reach up to 59% and 51%, respectively.

Keywords: Single Source Shortest Path, Δ -Stepping Algorithm, Relaxation Count Optimization, Shortest Path Tree, GPU Parallelization

目 录

摘 要.....	I
ABSTRACT.....	III
目 录.....	V
符号和缩略语说明.....	VII
第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.3 本文研究内容.....	6
1.4 后续章节安排.....	7
第二章 单源最短路径问题概述.....	8
2.1 预备知识.....	8
2.2 单源最短路径问题.....	12
2.3 三种经典单源最短路径算法.....	13
2.4 CUDA 并行框架介绍.....	18
2.5 本章小结.....	20
第三章 Δ -Stepping 算法改进及并行化介绍.....	21
3.1 Δ -Stepping 算法性能分析.....	21
3.2 松弛次数优化.....	22
3.3 基于最短路径树确定 Δ 值.....	25
3.4 算法的并行化.....	26
3.5 本章小节.....	28
第四章 数值实验与结果分析.....	29
4.1 实验准备.....	29
4.2 经典算法对比分析.....	30
4.3 松弛优化算法效果分析.....	33
4.4 Δ 参数值的优化效果分析.....	37
4.5 并行实验分析.....	39

目 录

4.6 真实图实验分析	40
4.7 本章小结	43
第五章 总结与展望	44
5.1 总结	44
5.2 展望	45
参考文献	46
致 谢	50

符号和缩略语说明

$G(V, E, W)$	以 V 为顶点集、 E 为边集和 W 为权值集的图 (Graph G with vertex set V , edge set E , and weight set W)
$ V = n$	表示图的顶点数 (The number of vertices of the graph)
$ E = m$	表示图的边数 (The number of edges of the graph)
$deg = \frac{m}{n}$	图的平均度数 (The average degree of the graph)
$d(v)$	表示顶点 v 的最短路径距离 (The shortest path distance of vertex v)

第一章 绪论

1.1 研究背景及意义

图论作为数学的一个重要分支,专注于研究由顶点和连接它们的边构成的图结构.由于图论研究兼具理论意义及应用价值,因此受到学术界及工业界的广泛关注.其中,最短路径问题作为图论研究的经典问题之一,旨在寻找图中两个顶点之间的最短距离.由于图中每条边的权重可以代表空间距离,也可以表示燃油消耗、行程费用等,所以最短路径算法在众多领域中都发挥着关键作用,例如在交通管控^[1]中,可协助规划最佳线路;在通信网络^[2]中,可确定数据包的最佳传输路径;在社交网络^[3]中,有助于理解社交关系和信息传播途径;在生物学^[4]中,支持研究神经网络模型和分子路径.

根据最短路径问题求解的不同,传统的最短路径问题可分为以下三类:点对点最短路径问题,单源最短路径问题以及全源最短路径问题.点对点最短路径问题主要解决给定起点和终点的情况,计算两顶点间的最短距离;单源最短路径问题则解决以一个特定顶点为起点,计算其到所有顶点的最短路径;全源最短路径问题解决图中任意两个顶点之间的最短路径.

近年来,随着科技的迅速发展,人们对于地图最优路径规划等方面提出了更高的要求.一方面,随着人们出行需求的日益增长,地图数据量呈现出爆炸式增长的趋势.以高德地图为例,其数据库包含了数十亿地点(点数)和数以千万计的道路信息(边数),且这些数据仍在不断更新和扩充,这对最短路径算法处理数据提出了更高的挑战.同时,用户对路径规划的响应时间要求日益严苛,特别是在实时导航领域,对时间精度的要求已进入微秒级.传统的最短路径算法在运行时间上已无法满足用户的需求.另一方面,计算能力发展迅猛,数据存储从传统的集中式存储逐渐发展到分布式存储系统,以满足当下大规模数据的存储与处理需求. GPU 硬件也在这几年间经历了高速发展,以 RTX 3080 (Ampere 架构) 和 GTX 780 (Kepler 架构) 为例, RTX 3080 的 CUDA 核心数量增加了约 3.8 倍,显存带宽增加了约 2.6 倍,为最短路径并行计算速度提升提供了更有力的保证.

综上所述,随着用户出行需求的持续攀升和对响应时间的不断缩短,加之计算能力的迅猛进步为最短路径数据处理提供了更为强大的支撑.因此,最短路径算法亟需进行优化和创新,以充分利用计算能力的提升,更好地满足现代社会对于高效、准确路径规划的需求.

1.2 国内外研究现状

最短路径是图论中一个重要的研究课题. 随着研究的深入, 算法得到了不断的改进和完善. 本节接下来分别对单源最短路径问题, 全源最短路径问题, 点对点最短路径问题展开论述, 表1-1列出了当前国内外最短路径主要算法优化及其效果.

表 1-1 国内外主要算法优化及其效果

最短路径问题	算法	主要结构和技术	效果 (时间复杂度)
单源最短路径问题	Dijkstra 算法	Fibonacci 堆	$O(m + n \log n)$
		Fusion Trees	$O(m + \frac{n \log n}{\log \log n})$
		radix 堆与 Fibonacci 堆结合	$O(m + n \sqrt{\log C})$
	Bellman-Ford 算法	动态逼近优化	$O(m)$
	Δ -Stepping 算法	动态调整步长	$O((m + n\rho) \log n)$ (工作量)
全源最短路径问题	Floyd-Warshall 算法	对所有顶点应用 Dijkstra 算法	$O(mn + n^2 \log n)$
		EREW PRAM 模型	$O(\frac{f(n)}{p} + \log n \log n)$
		CRCW PRAW 模型	$O(\frac{n^3}{p} + \log nt)$
		邻接矩阵分割成矩形子矩阵	$O(\frac{n^3 \sqrt{\log \log n}}{\sqrt{\log n}})$
点对点最短路径问题	A* 算法	动态数据 FIFO 属性	应用至动态网络
	蚁群算法	采取双曲正切函数作为搜索函数 结合模拟退火算法或者目标驱动重力	确定初始方向 加快收敛速度

1.2.1 单源最短路径问题

单源最短路径问题是寻找图中固定起点到其他所有顶点的最短距离. 在实际生活中, 可以理解为城市是图的顶点, 而城市道路则是图的边. 每条边都有关联的距离, 表示行程的开销. 解决单源最短路径问题则为找到从出发地到目的地的最短路径, 虽然这样的路径可能不唯一, 但是可以从所有路径中寻找开销最小的路径. Dijkstra 算法和 Bellman-Ford 算法正是解决单源最短路径问题中最为经典的算法. Dijkstra 算法最初是由荷兰计算机科学家 Edsger Wybe Dijkstra^[5] 于 1959 年在其论文中提出, 该算法主要是利用贪心算法的思想, 通过不断修正边的最短距离来找到最短路径, 同时也进一步指出仅存储一部分边或路径数据来降低内存需求, 而不需要同时存储所有边的数据. 对于大型稀疏图而言, 一般矩阵存储容易造成空间浪费, 采用链表结构存储数据可以使算法性能得到很大改善. 之后 Fredman 和 Tarjan^[6] 扩展了由 Vuillemin 和 Brown 提出的二叉堆数据结构, 采用 Fibonacci 堆结构将非负单源最短路径问题改进至 $O(m + n \log n)$. 1990 年, Fredman 和 Willard^[7] 提出 Fusion Trees 对于动态数据结构的改进, 使得 Dijkstra 算法的时间复杂度进一步

减少至 $O(m + \frac{n \log n}{\log \log n})$. Ravindra 等人^[8] 提出新的数据结构 radix 堆, 并与 Fibonacci 堆进行结合, 成功实现更优的时间复杂度 $O(m + n\sqrt{\log C})$, 其中 C 是界限. Deng 等人^[9] 基于模糊数的渐进均值积分对 Dijkstra 算法进行扩展以解决实际生活中具有模糊弧长的最短路径问题. 在 2023 年, 为更好解决用户实时公共交通路线查询, Aryo, Agi 和 Muhammad^[10] 通过使用 Douglas-Peucker 线简化算法来减少 Dijkstra 算法处理顶点数, 从而降低整体运行时间和内存占用. 随着数据结构与并行化技术的持续进步, Dijkstra 算法不断优化, 但其在处理负权图时仍面临挑战.

Bellman-Ford 算法是单源最短路径的另一经典算法, 可以处理带负权边的图并判别是否存在负权回路. 该算法最初是由 Richard^[11] 提出, 并在文献中^[12] 给出其收敛性说明, 其主要思想是对所有的边进行 $n - 1$ 次松弛操作, 但时间复杂度高达 $O(mn)$. Yen^[13] 于 1970 年针对包含负权重图的算法进行改进, 证明不存在负循环的情况下, 最多需要 $\frac{1}{2}M(n - 1)(n - 2)$ 次计算; 存在负循环的情况下, 需要 $\frac{1}{2}n(n - 1)(n - 2)$ 次计算, 其中 $1 < M \leq n - 1$. Michael 和 David^[14] 给出了 Bellman-Ford 算法的变体, 对顶点进行随机排列并在每次迭代中利用这种随机顺序处理顶点, 相比 Yen^[13] 提出的算法松弛步数减少了 $\frac{2}{3}$. Duan^[15] 于 1994 年提出了关于单源最短路径的一种新的算法 SPFA, 时间复杂度为 $O(m)$, 而且该算法对图中边的权重没有限制. Mani 等人^[16] 于 2021 年提出基于梯形图片模糊数的 Bellman-Ford 算法, 用于在网络中找到最短图片模糊路径.

2003 年, Meyer 和 Sanders^[17] 首次提出 Δ -Stepping 算法, 该算法是对 Dijkstra 算法和 Bellman-Ford 算法的结合, 当 Δ 取值为 1 时, 为 Dijkstra 算法的变体; 当 Δ 取值为无穷大时, 近似为 Bellman-Ford 算法. 2016 年, Blelloch 等人^[18] 提出动态调整步长 Radius-Stepping, 该算法工作量为 $O((m + n\rho) \log n)$, 深度为 $O(\frac{n}{\rho} \cdot \log n \log(\rho L))$, 其中 ρ 是步长, L 是图中边的最大权重. 之后, Dhulipala 等人^[19] 为有效支持顶点分桶操作, 开发 Julianne 框架用于维护各个桶顶点的插入和删除操作, 成功实现在几秒钟内处理拥有数十亿条边的图表, 几分钟内处理拥有数千亿条边的图表. 2021 年, Dong 等人^[20] 为解决 Δ -Stepping 算法效率高度依赖于参数 Δ , 提出 “stepping” 通用算法框架并给出了一系列现有算法的改进边界.

随着数据量爆炸式增长, 最短路径算法并行化的实现与改进迫在眉睫. Lei 等人^[21] 为处理大规模图, 提出基于 “eager” 变种的 Dijkstra 算法并行 FPGA 的实现方案, 可以达到 CPU 实现 5 倍加速比. Federico 和 Nicola^[22] 基于 Kepler 架构提出 H-BF 算法, 通过优化内存访问, 避免了缓存污染问题, 并且利用了 GPU 的 L1/L2 缓存来提高访问速度, 从而获得了更好的性能表现. Venkatesan 等人^[23] 基于 Bellman-Ford 和 Δ -Stepping 算法, 采用包括边分类和方向优化等剪枝技术使得

负载均衡从而大大提高算法处理效率. 2021 年, Yu, Wang 和 Luo^[24] 提出一种基于边缘围栏策略的以路径为中心的单源最短路径算法, 该算法对偏斜度分布图表现优异.

由于图的点、边和权重在实际生活中存在动态变化, 如果仍旧利用静态单源最短路径算法重新计算, 将会耗费大量计算资源. 所以后续研究多集中于求解动态图的最短路径. 2014 年, Ding 和 Lin^[25] 首次考虑动态图上非负权重的单源最短路径, 提出局部搜索算法 PSAL, 最坏情况下期望更新时间为 $O(n + \frac{n^2 \log n}{m})$. Riazzi 和 Srinivasan^[26] 进而针对动态图提出利用先前快照计算结果以减少数据开销的 SSSPIncJoint 分布式计算方法, 可达 2.2 倍加速比. 2023 年, Luay, Eyad 和 Sundus^[27] 引入回溯优先队列数据结构, 使得 Dijkstra 算法具有动态性, 可根据最新的网络变化数据自动找到并更新最佳路径.

1.2.2 全源最短路径问题

全源最短路径是对图中所有顶点对求解最短路径, 相比单源最短路径, 不仅对算法准确度和时间复杂度提出更高要求并且对计算机算力要求较高. 1962 年, Robert^[28] 首次提出 Floyd-Warshall 算法, 其原理主要是采用动态规划思想不断更新顶点间的路径距离从而求解图中所有顶点对之间的最短距离, 其时间复杂度为 $O(n^3)$. 但是该算法通过三重循环求解最短路径, 会造成算法时间复杂度较高, 所以也有研究采用对所有顶点都应用 Dijkstra 算法, 其算法时间复杂度为 $O(mn + n^2 \log n)$, 虽然时间比 Floyd-Warshall 短, 但是 Dijkstra 算法不能处理负权图. 1977 年, Johnson^[29] 提出重新赋权值的方法将带负权的图转化为权值非负的图, 再对每个顶点使用 Dijkstra 算法成功解决含负权图的全源最短路径问题. Han 和 Pan^[30] 于 1992 年对并行算法在不同模型下计算有向图的所有顶点间最短路径的时间复杂度展开研究, 其中 EREW PRAM 模型下的时间复杂度为 $O(\frac{f(n)}{p} + \log n \log n)$, 随机化 CRCW PRAM 模型下的时间复杂度为 $O(\frac{n^3}{p} + \log n)$, 其中 $f(n)$ 是小于 n^3 的函数. 1992 年, Tadao^[31] 采用将邻接矩阵分割成矩形子矩阵, 并使用查表法进行矩阵相乘等操作, 算法复杂度降至 $O(\frac{n^3 \sqrt{\log \log n}}{\sqrt{\log n}})$. 2016 年, Han 和 Tadao^[32] 继续改进, 将算法的时间复杂度成功降低至 $O(\frac{n^3 \log \log n}{\log^2 n})$.

在解决大规模图的全源最短路径问题时, 传统的顺序算法, 比如 Floyd-Warshall 和 Johnson 等效率比较低, 因此需要对其采用并行化处理, 主要有两种方法: 一是对单源最短路径所有顶点进行并行化, 二是利用并行框架并行化全源最短路径算法. 2006 年, U. Bondhugula 等人^[33] 提出借用 FPGA 技术针对解决有向图中全源最短路径 Floyd-Warshall 算法进行改进. Gary 和 Joseph^[34] 于 2008 年利用 CUDA

对 Floyd-Warshall 算法进行并行化处理, 将图划分为多个数据块并且基于数据块进行迭代, 高效利用共享内存缓存, 在 10000 多个顶点的图上加速比约为 25. 2016 年, Schoeneman 和 Zola^[35] 提出在 Apache Spark 分布式集群上求解全源最短路径, 并且性能最佳的求解器可以在一个 1024 核的集群上处理包含 20 万个顶点的全源最短路径问题. Yang 等人^[36] 提出在分布式集群的基础上, 将 Floyd-Warshall 算法和 Dijkstra 算法进行结合进而提出一种新的快速算法, 该算法可达 16.79 倍加速比. Muhammad, Karl-Heinz 和 Asad^[37] 提出基于热带矩阵将 Floyd-Warshall 算法求解二部图最短路径问题转化成热带矩阵积, 并且与串行的 Floyd-Warshall 算法相比, 加速比达到 274 倍. 为了最大化利用 GPU 架构的高并行效率, Sao 等人^[38] 通过流水线操作和异步操作减少通信成本, 并且采用数据分离模型, 达到理论峰值的 80% 的并行效率.

1.2.3 点对点最短路径问题

在单源最短路径确定终点的情况下, 点对点最短路径可以看成是单源最短路径的特例. 因此, 直接关于点对点最短路径的研究比较少. 但是为快速搜索点对点最短路径, Peter E. Hart^[39] 于 1967 年提出 A* 算法, 它利用启发式函数来估计起点到目标顶点的距离, 通过评估路径的估计成本来引导搜索, 通常比 Dijkstra 算法更快找到最短路径, 且适用于多种场合, 如游戏中的路径规划、地图导航等. Chabini 和 Lan^[40] 提出将 A* 算法扩展到动态网络中, 利用动态数据 FIFO 属性和动态网络的时间扩展隐式表示对 A* 算法进行动态适应, 并提出了关于最小行进时间的改进下界. 2020 年为解决 A* 算法在某些特定条件下生成的路径并非最短的问题, Zhu、Luo 和 Yan^[41] 提出了一种基于两点间最短线段原理对传统 A* 算法进行优化的新方法, 该算法成功提高路径规划的效率, 所生成的路径长度相较于当时其他算法均有所缩短. M.Dorigo, V. Maniezzo 和 A. Colomi^[42] 于 1996 年提出蚁群算法, 其主要想法来自于蚂蚁在迷宫里寻找目的地时是依靠前面蚂蚁选择路径时留下的信息素浓度, 浓度越高的路径蚂蚁越有可能选择它作为下一步的前进方向. 之后, 2016 年易正俊, 李勇霞和易校石^[43] 对蚁群算法初始方向上采取双曲正切函数作为搜索函数, 每次会偏向信息素浓度较高的方向, 从而保证搜索的高效性. 朱经纬等人^[44] 针对二次分配问题, 将模拟退火算法于蚁群算法进行结合, 并使用较小的温度系数的“反火”方法, 使得算法具有更高的稳定性和更快的收敛速度. 为解决传统蚁群算法移动机器人路径规划收敛速度慢、易陷入局部最优解等问题, Chen 等人^[45] 提出一种基于目标驱动重力的改进蚁群算法, 该算法将一阶重力函数替换为高阶指数形式, 从而使得该算法具有更快的收敛速度和更高的全局搜索能力.

由于传统算法串行实现已经不太能满足大规模图的点对点求解最短路径, 因

此有许多学者对传统算法进行并行化研究. V.Scott Gordon 和 L. Darrell Whitley^[46] 于 1993 年对并行遗传算法进行研究发现, 配对遗传算法性能往往优于单一实现. 2011 年, 赵辉和徐俊刚^[47] 针对大规模 TSP 问题下收敛速度慢提出利用共享内存编程的 OpenMP 设计并行蚁群算法, 使 CPU 利用率从 50% 到达 100%. 此外, Takuya, Yoichi 和 Yuichi^[48] 提出在对每个顶点进行广度优先搜索时, 会进行剪枝处理从而优化搜索范围.

1.2.4 存在的问题

尽管图的存储方式和最短路径算法一直在完善和改进, 但关于 Δ -Stepping 算法的研究仍存在以下问题:

- 常见的 Δ -Stepping 算法关于轻重边的定义是基于 Δ 为界限, 大于 Δ 为重边, 小于或等于 Δ 为轻边. 而且未考虑轻重边作用以及无效边的剔除, 从而造成松弛次数的浪费.
- 传统 Δ 值的确定方法采用随机选取的方式, 这种方法因缺乏针对性而在不同类型的图上适用性有限. 而且当 Δ 值相对图来说过小或者过大的话, 容易造成桶利用率不足.
- 现存并行方案主要在 CPU 上基于 MPI 框架, 通信时间较长, 对 GPU 上的并行化考虑较少.

1.3 本文研究内容

前文主要介绍了点对点最短路径问题、单源最短路径问题及全源最短路径问题的研究与发展. 由于单源最短路径固定终点即可解决点对点最短路径问题, 多次计算可以求解全源最短路径, 且关于单源最短路径算法的研究及应用也比较多, 所以本文选取单源最短路径为研究方向. 单源最短路径经典算法主要是 Dijkstra 算法, Bellman-Ford 算法和 Δ -Stepping 算法. 由于 Δ -Stepping 算法在特殊情况下是 Dijkstra 算法和 Bellman-Ford 算法的变体, 并且该算法在求解大规模单源最短路径方面上时间开销比较小, 也具有易并行化方面的优点. 所以, 本文针对 Δ -Stepping 算法存在的问题主要做了以下三个方面的改进:

- 针对松弛次数的优化, 一方面对 Δ -Stepping 算法原有的轻边重边的定义进行修改, 另一方面对桶内边进行细化, 找出无效边和叶顶点从而减少无效松弛次数和跳过松弛条件直接进行距离更新.
- Δ 初始值的确定是依据最短路径树的平均距离与平均深度的商值, 因其可以有效平衡长距离最短路径和短距离最短路径的探索, 有利于在算法的初期阶

段快速收敛, 更好地覆盖图中的各个部分, 而不是过于偏重于某一特定区域.

- 利用 CUDA 并行框架对 Δ -Stepping 算法进行混合并行优化, 实现了清桶阶段的顶点并行处理和松弛阶段的边分配并行计算.

1.4 后续章节安排

本文后续章节安排具体如下:

第二章主要对单源最短路径问题进行概述, 首先介绍图的预备知识, 并详细分析三种经典的单源最短路径算法: Dijkstra 算法、Bellman-Ford 算法以及 Δ -Stepping 算法, 最后介绍了 CUDA 并行框架.

第三章是对 Δ -Stepping 算法进行性能分析并提出改进措施和并行化研究, 主要包括松弛次数优化, 通过最短路径树启发式确定 Δ 值以及基于 GPU 并行的优化.

第四章是数值实验部分, 在常见网络图上进行三大经典算法对比分析, 然后将 Δ -Stepping 原始算法与改进后的算法进行比较分析, 最后在不同真实图上分别对串行实现算法和并行实现算法两种方式进行实验分析.

第五章是结论与展望, 总结文章亮点和未来仍需改进的方向.

第二章 单源最短路径问题概述

本章主要是对单源最短路径预备知识进行简要介绍. 首先, 介绍相关图的基本定义、图存储格式和常见网络图; 其次, 给出单源最短路径问题定义; 然后, 展开介绍三种经典单源最短路径算法原理并分析优缺点; 最后, 介绍 CUDA 并行框架.

2.1 预备知识

2.1.1 图的定义

在图论中, 图的定义主要由顶点和边构成, 一般表示为 $G(V, E, W)$, 其中 V 是顶点集, E 是所有边的集合, W 是所有边权值的集合. 图的总顶点数一般用 $|V| = n$ 来表示, 总边数则用 $|E| = m$ 表示. 为了更清晰地阐述图的相关定义, 引入以下网络图进行说明, 具体如图2-1所示.

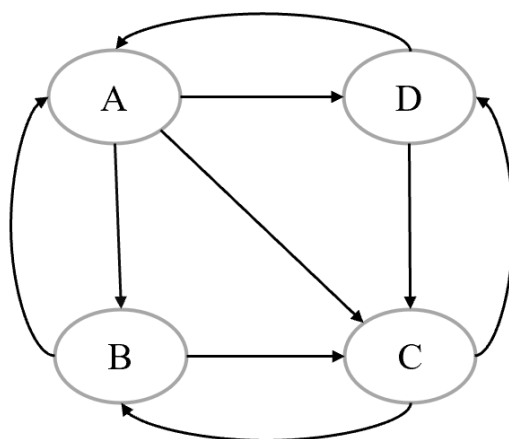


图 2-1 网络图示例

在图论中, 根据边的指向性, 可以将图分为有向图和无向图. 图2-1展示了一个简单的有向图示例, 边 (A, B) 表明是从点 A 到点 B , 如果边 (A, B) 没有方向指示, 则为无向图. 在有向图中, 顶点的出度表示该顶点发出的边的总数量, 顶点入度表示该顶点接收的边的总数量. 在无向图中, 由于没有方向指示, 按照定义顶点度数为与该顶点相连的边的总数.

如果图中任意一条边都附加权重, 则该图为有权图, 否则该图为无权图. 这里的权重可以是距离、时间、耗油量等. 通常有权图用 $G(V, E, W)$ 表示, 无权图则用 $G(V, E)$ 表示.

基于图中顶点个数与边的个数的数量关系,通常将图分为稀疏图和稠密图.一般来说,稀疏图是指边的数量相对于顶点数来说较少的图.本文设定以 $m = n \log n$ 作为图的分类标准,其中 m 表示边的数量, n 表示顶点的数量.当边的数量较少,即满足 $m < n \log n$ 的条件时,将该图视为稀疏图,这意味着图中顶点之间的连接相对稀疏,边的数量不多.反之,当边的数量较多,不满足上述条件时,则将该图分类为稠密图.

如无特殊说明,本文讨论及研究对象是权值非负的图.

2.1.2 图存储格式

在图论领域,图的存储方式经历了多次创新和改进,从最初的邻接矩阵存储到后来的邻接列表.不同的存储方式在空间利用上有显著差异.最简单的邻接矩阵存储需要 n^2 的空间,其中 n 为图的顶点数.当图的顶点数量不大时,这种方式并不会暴露出明显的问题.但是当顶点数量极大时,邻接矩阵存储可能导致巨大的空间占用,尤其是在稀疏图的情况下,可能造成空间浪费.

为了应对这个问题,研究者们提出了将二维邻接矩阵压缩成一维数组的方法来存储图的结构.其中,最常见的一维数组存储格式是 Coordinate (COO) 格式. COO 格式利用三个一维数组存储邻接矩阵中非零值的坐标信息,分别是按行存储的行坐标数组索引值、列坐标数组索引值和值数组.这种方式占用的空间为 $3m$, 其中 m 是非零权值边的数量.尽管 COO 格式有效地减少了存储空间的占用,但仍存在进一步改进的空间.

为了进一步压缩存储空间, Cormen 等人^[49] 提出了压缩稀疏行 CSR (Compressed Sparse Row) 格式. CSR 格式同样利用三个一维数组存储信息,但第一个数组存储的不是非零值的行坐标信息,而是前面行数非零值的总个数.这种方式在保持图结构信息的同时,进一步减少了存储空间的需求.在这种方式下,占用的空间约为 $2m + n + 1$. 图2-2中具体展示了邻接矩阵 C 转换成 CSR 格式存储,可以观察到 CSR 格式存储图并未改变图的拓扑结构,并且邻接矩阵和 CSR 格式可以互相转换.

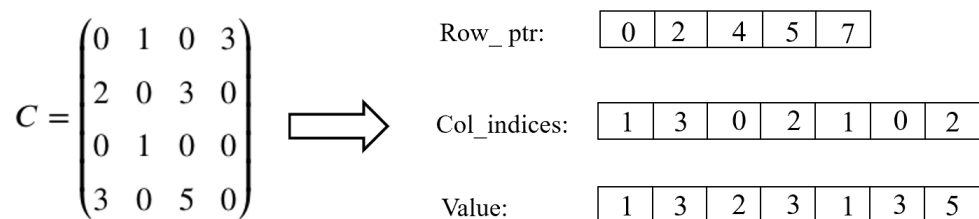


图 2-2 CSR 格式存储图

2.1.3 常见网络图

在日常生活中，事物可以抽象成图中的点与点之间的连接关系，形成网络图。网络图是图论的重要分支之一，与图的基本定义类似，由多个顶点和顶点之间的边组成，其中顶点表示实体，边表示实体之间的关系。在生活的许多领域中，如社交网络分析、交通规划、生物信息学和通信网络等，都可以应用网络图的理论和方法用于解决一些抽象的问题。以下介绍常见的四种网络图。

2.1.3.1 规则网络图

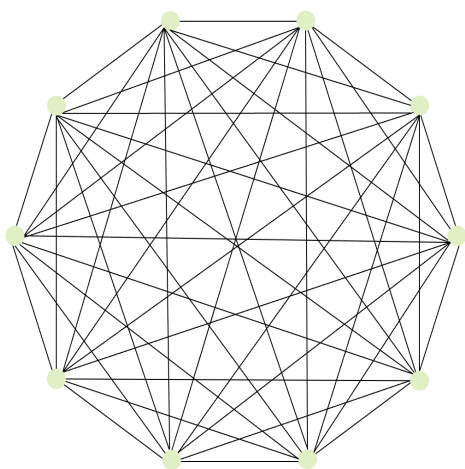


图 2-3 规则网络图示例

规则网络图的特点在于其顶点之间连接的规则性。在这种图中，每个顶点的度都是相同的，即每个顶点的邻居顶点个数都是相同的，这使得图具有一定的结构和对称性。以图2-3为例，这是一个规则网络图，其中每一个顶点的度数都是 9，总共包含 10 个顶点。

2.1.3.2 ER 随机网络图

ER 随机网络图 (Erdős-Rényi Random Graph) 是由数学家 Erdős 和 Rényi^[50] 在 1959 年提出的一种经典图论模型，其定义是基于概率空间和随机边的连接。ER 随机网络图的性质主要取决于给定的边的连接概率 p ，即图中任意两个顶点之间存在边的概率。当图中顶点数一定时，如果给定的边的连接概率 p 比较小，则图比较稀疏；如果给定的边的连接概率 p 比较大，则图比较稠密。以图2-4为例，这是一个 ER 随机网络图，其中边的连接概率是 0.5，总共包含 10 个顶点。

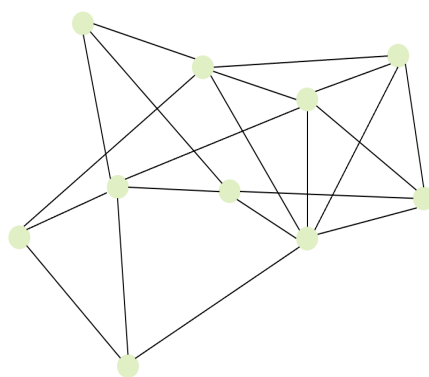


图 2-4 ER 随机网络图示例

2.1.3.3 WS 小世界网络图

WS 小世界网络图这一概念最早是由美国社会学家 Stanley Milgram 提出的。起初，为刻画人与人之间的社交网络关系网，他进行了“连锁信”实验研究，该实验发现：通过一系列信件的中转，为将信成功送达指定的人，一般需要 6 个中间人。这项实验表明，人与人之间的社交网络关系网具有“小世界”现象，即人与人之间的关系网络具有短路径的特点。之后这个概念被扩展成小世界网络模型，成为复杂网络研究的一个重要方向。WS 小世界网络图一般被认为是规则网络图向 ER 随机网络图的过渡。因此 WS 模型实质上是通过在原有部分规则性结构的基础上引入随机性，模拟真实社交网络的小世界属性。以图 2-5 为例，这是一个 WS 小世界网络图，顶点数为 10，顶点最大度数为 6，其中边连接概率是 0.6。

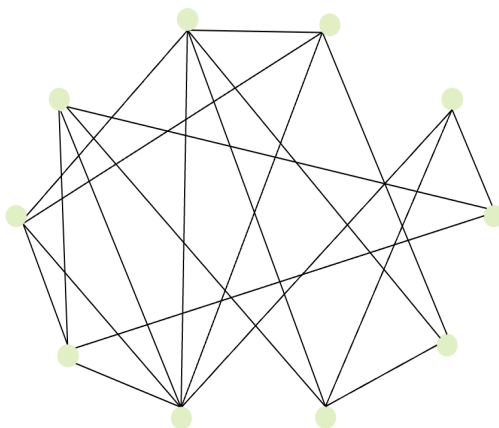


图 2-5 WS 小世界网络图示例

2.1.3.4 BA 无标度网络图

BA (Barabási-Albert) 无标度网络图是由 Barabási 和 Albert^[51] 于 1999 年提出的。该网络图是基于无标度网络理论，其网络结构不同于规则网络，它的拓扑结构

是随时间变化的,其中顶点也具有不同的度数.无标度网络 (Scale-Free Network) 是指图中顶点度数按照幂律分布,具体而言,大多数顶点仅拥有少量的邻接顶点,即它们的连接数相对较低;而少数顶点则拥有极高的度数,即它们与网络中大量的其他顶点相连接.该模型的核心思想是网络中新加入的顶点连接到已有顶点的概率与已有顶点的度数成正比,致使原有网络中的一些顶点通过原有度数较高叠加形成网络中的“枢纽顶点”.所以,BA 无标度网络的提出为生活中一些复杂网络的形成提供了一个全新的解释框架,对生物学、人际关系网和互联网等领域都具有重要意义.以图2-6为例,这是一个 BA 无标度网络图,共 15 个顶点,每次加入一条边.

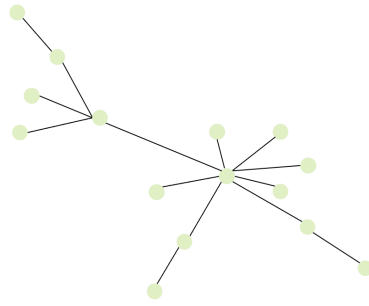


图 2-6 BA 无标度网络图示例

2.2 单源最短路径问题

定义 2.1: 在图 $G(V, E, W)$ 中,如果边 $(v_i, v_{i+1}), (v_{i+1}, v_{i+2}), (v_{i+2}, v_{i+3}), \dots, (v_{j-1}, v_j)$ 存在,则 $l = \langle v_i, v_{i+1}, v_{i+2}, \dots, v_j \rangle$ 就是点 v_i 到点 v_j 的一条完整路径.其中路径 l 距离为

$$w(l) = \sum_{i=1}^{j-1} w(v_i, v_{i+1}).$$

定义 2.2: 最短路径是指从点 v_i 到点 v_j 的路径中,使得路径上所有边权重之和最小的路径.如果存在多条最短路径,则可以选择其中任意一条作为最短路径.最短路径长度计算公式如下:

$$d(v_i, v_j) = \min(w(L)),$$

其中, L 是从点 v_i 到点 v_j 的所有路径的集合, $w(L)$ 是路径集合 L 中所有路径的权重的集合.

定义 2.3: 设 u 和 v 是图 $G(V, E, W)$ 中两个顶点, $w(u, v)$ 表示顶点 u 到顶点 v 的边的权重, $d(u)$ 和 $d(v)$ 分别表示源点 s 到顶点 u 和顶点 v 的最短距离.如果 $d(u) + w(u, v) < d(v)$, 则更新 $d(v)$ 为 $d(u) + w(u, v)$. 这一过程就是松弛操作.

单源最短路径问题是求解源点到图中所有顶点的最短路径,求解思路如下:初

始令其他点到源点距离为无穷大, 之后通过不断迭代对距离进行松弛更新操作, 直到所有顶点的距离都不再发生改变即求解结束.

2.3 三种经典单源最短路径算法

随着对单源最短路径问题研究的不断深入, 算法领域相继涌现出多种高效解决方案. 其中, Dijkstra 算法和 Bellman-Ford 算法作为早期的经典方法, 各自在特定场景下展现出独特的优势. 然而, 为了进一步提升算法性能并适应更广泛的场景需求, 研究者们提出了结合 Dijkstra 算法和 Bellman-Ford 算法优势的 Δ -Stepping 算法. 因此, 本章旨在介绍 Dijkstra 算法、Bellman-Ford 算法和 Δ -Stepping 算法各自的优缺点.

2.3.1 Dijkstra 算法

Dijkstra 算法是单源最短路径经典算法之一, 该算法主要利用了贪心算法的思想, 通过不断修正边的最短距离来找到最短路径. 具体而言, 从源点开始, 每次选择当前距离中的最短距离作为本轮开始点进行探索, 并以此更新其邻居顶点的最短距离, 直到所有顶点都被访问且最短距离不再更新为止, 算法结束. 因此, 该算法的时间复杂度为 $O(n^2)$. 此外, 该算法擅长处理稀疏图上的单源最短路径问题, 在稠密图或者图的密集区域上表现一般. Dijkstra 算法的伪代码如算法2.1所示.

算法 2.1 Dijkstra Algorithm

```

1: Input: Graph  $G = (V, E, W)$ , source vertex  $s$ 
2: Output: Shortest distances  $d(v)$  for all  $v \in V$ 
3: Initialization:
4: for each  $v \in V$  do
5:    $d(v) \leftarrow \infty$ 
6: end for
7:  $d(s) \leftarrow 0$ 
8:  $T \leftarrow V \setminus s$ 
9: while  $T$  is not empty do
10:   Min:  $u \leftarrow \{v : v \in T \text{ and } \forall c \in T, d(v) \leq d(c)\}$ 
11:   if  $d(u) = \infty$  then
12:     break
13:   end if
14:   Remove  $u$  from  $T$ 
15:   Relax:
16:   for each  $(u, v) \in E$  do
17:     if  $d(u) + w(u, v) < d(v)$  then
18:        $d(v) \leftarrow d(u) + w(u, v)$ 
19:     end if
20:   end for
21: end while
22: return  $d$ 

```

接下来通过图2-7详细阐述 Dijkstra 算法求解单源最短路径步骤:

- (1) 首先进行初始化操作, 将源点 A 距离设置为 $d(A) = 0$, 所有顶点进入优先级队列 $T = \{A, B, C, D, E\}$, 除源点外其余顶点由于距离未知所以临时最短距离设置成无穷大. 初始时最小距离是源点, 于是对源点的邻居顶点进行距离更新, 即 $d(B) = 1$, $d(D) = 3$, $d(E) = 10$.
- (2) 此时优先级队列中距离最小的顶点是 B , 对其邻居顶点进行距离更新, 即 $d(C) = 6$, 并把 B 从优先列表进行剔除. 这时优先队列是 $T = \{C, D, E\}$, 选取优先级队列 T 中距离最小的顶点 D , 并对其邻居顶点进行距离更新, 即 $d(C) = 5$, $d(E) = 9$, 并把顶点 D 从优先级队列 T 中剔除. 重复上述操作, 直至优先级队列 T 为空, 算法结束. 最后求得以 A 为源点的单源最短距离为 $d(A) = 0$, $d(B) = 1$, $d(C) = 5$, $d(D) = 3$, $d(E) = 6$.

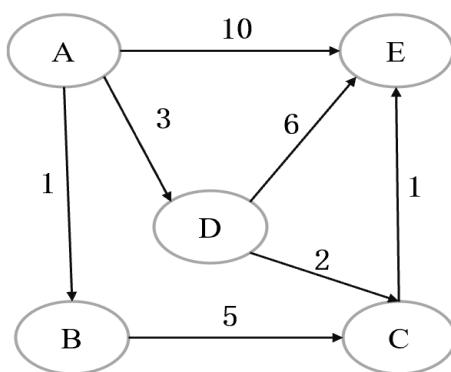


图 2-7 Dijkstra 算法示例图

2.3.2 Bellman-Ford 算法

Bellman-Ford 算法是解决单源最短路径问题的另一经典算法, 其核心思想基于动态规划. 该算法通过多次循环迭代逐步寻找最短路径. 对于每一条边 (u, v) , 算法尝试通过中间顶点 u 来更新顶点 v 的最短路径. 由于中间顶点最多可能有 $n - 1$ 个, 因此迭代次数不超过 $n - 1$ 次. 如果算法出现循环超过 $n - 1$ 次, 最短距离仍然发生改变, 则表明图中存在负环. 因此, 该算法具有检测负环的优点. 同时, 也正因为多轮迭代寻找最短路径, Bellman-Ford 算法在并行方面也具有优点, 这一点在分布式系统和 GPU 环境中尤为明显. 但是, 由于每个顶点都需要进行 $n - 1$ 次松弛操作, 总的时间复杂度会较高, 为 $O(mn)$. 这使得该算法在大规模图上的应用受到一定限制. Bellman-Ford 算法的伪代码具体如算法2.2所示.

以下通过图2-8详细阐述 Bellman-Ford 算法求解单源最短路径步骤:

- (1) 首先进行常规初始化操作, 将源点 A 距离设置为 $d(A) = 0$, 其余顶点由于距

离未知则将其临时最短距离设置成无穷大.

- (2) 随机从所有边中选取一条边开始, 比如边 (A, D) , 可以发现 $d(A) + w(A, D) < d(D)$, 则对 $d(D)$ 距离进行更新操作, 依次松弛所有边.
- (3) 重复 (2) 的步骤, 更新迭代 $n - 1$ 次.
- (4) 检测负权环, 在完成 $n - 1$ 次迭代后, 如果最短距离仍发生改变, 那么图中存在负权环. 最后, 求得以 A 为源点的单源最短距离为 $d(A) = 0$, $d(B) = 1$, $d(C) = 3$, $d(D) = 0$, $d(E) = 4$.

算法 2.2 Bellman-Ford Algorithm

```

1: Input: Graph  $G = (V, E, W)$ , source vertex  $s$ 
2: Output: Shortest distances  $d(v)$  for all  $v \in V$ 
3: Initialization:
4: for each  $v \in V$  do
5:    $d(v) \leftarrow \infty$ 
6: end for
7:  $d(s) \leftarrow 0$ 
8: for  $i \leftarrow 1$  to  $|V| - 1$  do
9:   for all edges  $(u, v) \in E$  do
10:    if  $d(u) + w(u, v) < d(v)$  then
11:       $d(v) \leftarrow d(u) + w(u, v)$ 
12:    end if
13:  end for
14: end for
15: for all edges  $(u, v) \in E$  do
16:  if  $d(u) + w(u, v) < d(v)$  then
17:    return "Negative weight cycle"
18:  end if
19: end for
20: return  $d$ 

```

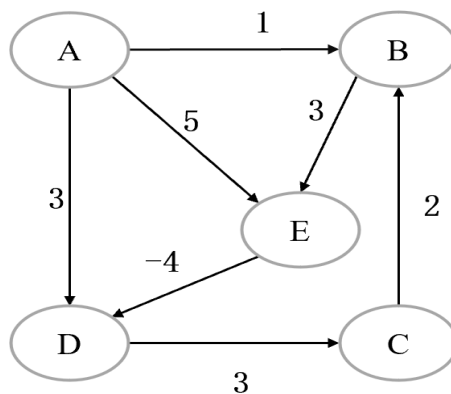


图 2-8 Bellman-Ford 算法单源最短路径示例

2.3.3 Δ -Stepping 算法

Dijkstra 算法的时间复杂度为 $O((m+n)\log n)$ ，但在大规模图上表现不佳，因其时间复杂度与顶点数 n 相关. Bellman-Ford 算法虽然在求解单源最短路径问题时具有较好的并行性，但其时间复杂度为 $O(mn)$ ，在大规模图上效率仍然较低.

为了解决上述问题，Meyer 和 Sanders^[17] 于 2003 年首次提出了 Δ -Stepping 算法，这一重要算法在学术界引起了广泛关注. 该算法首次引入了“桶”的概念，通过将各个顶点按照一定规则放置在不同的桶中，依次进行松弛操作，直到所有桶都清空，算法结束. 其中 Δ 参数在该算法中发挥了重要作用，不仅确定桶的宽度，还对顶点的出边进行轻边和重边的分类. 具体而言，当边的权重 $w(u, v)$ 小于等于 Δ 时，该边为轻边，而当 $w(u, v)$ 大于 Δ 时，该边为重边. 这一分类的原因在于更新顶点的临时最短距离时，轻边对距离的更新影响较为显著.

算法 2.3 Δ -Stepping Algorithm

```

1: Input: Graph  $G = (V, E, W)$ , source vertex  $s$ 
2: Output: Shortest distances  $d[v]$  for all  $v \in V$ 
3: Initialization:
4: for each  $v \in V$  do
5:    $heavy(v) \leftarrow \{(u, v) \in E : w(u, v) > \Delta\}$ 
6:    $light(v) \leftarrow \{(u, v) \in E : w(u, v) \leq \Delta\}$ 
7:    $d(v) \leftarrow \infty$ 
8: end for
9:  $relax(s, 0)$ 
10:  $i \leftarrow 0$ 
11: while  $B \neq \emptyset$  do
12:    $S \leftarrow \emptyset$ 
13:   while  $B[i] \neq \emptyset$  do
14:     (b1) Add To Request:
15:      $Req \leftarrow \{(w, d(u) + w(u, v)) : u \in B[i] \wedge (u, v) \in light(u)\}$ 
16:      $S \leftarrow SUB[i]$ 
17:      $B[i] \leftarrow \emptyset$ 
18:     (c1) Relax:
19:     for each  $(u, x) \in Req$  do
20:        $relax(u, v)$ 
21:     end for
22:   end while
23:   (b2) Add To Request:
24:    $Req \leftarrow \{(w, d(u) + w(u, v)) : v \in S \wedge (u, v) \in heavy(u)\}$ 
25:   (c2) Relax:
26:   for each  $(u, x) \in Req$  do
27:      $relax(u, v)$ 
28:   end for
29:    $i \leftarrow i + 1$ 
30: end while

```

算法 2.4 Relax Operation

```

1: Input: relax( $u, v$ )
2: Output: update  $d(v)$ , bucket index
3: if  $d(u) + w(u, v) < d(v)$  then
4:    $B[\lfloor d(v)/\Delta \rfloor] \leftarrow B[\lfloor d(v)/\Delta \rfloor] \setminus v$ 
5:    $B[\lfloor (d(u) + w(u, v))/\Delta \rfloor] \leftarrow B[\lfloor (d(u) + w(u, v))/\Delta \rfloor] \cup \{v\}$ 
6:    $d(v) \leftarrow d(u) + w(u, v)$ 
7: end if

```

Δ -Stepping 算法的伪代码如2.3所示. 其中松弛操作与 Dijkstra 算法和 Bellman-Ford 算法有些不同, Δ -Stepping 算法增加了对顶点桶编号的更新处理, 其伪代码如2.4所示.

以下通过图2-9详细阐述 Δ -Stepping 算法求解单源最短路径的具体步骤:

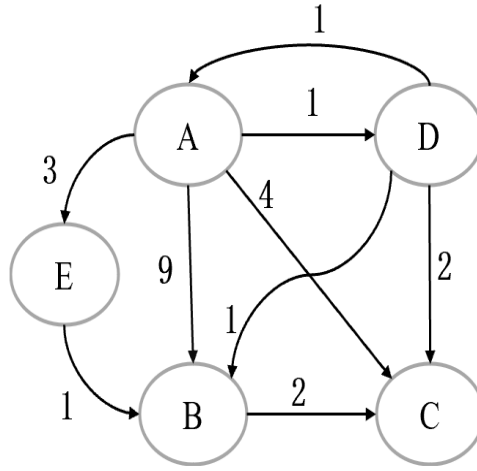


图 2-9 Δ -Stepping 算法单源最短路径示例图

- (1) 首先进行常规初始化操作, 将源点 A 距离设置为 $d(A) = 0$, 其余顶点的临时距离设置成无穷大, 且 Δ 取值为 2.
- (2) 开始时, 只有 $B[0]$ 桶中存在点 A . 则对 $B[0]$ 桶进行清桶操作, 将点 A 放入到集合 S 中, 发现点 A 具有轻边 (A, D) , 松弛更新后的最短距离为 $d(A) = 0$, $d(B) = \infty$, $d(C) = \infty$, $d(D) = 1$, $d(E) = \infty$.
- (3) 由于点 D 落入 $B[0]$ 桶, 则继续清桶. 将点 D 加入到集合 S , 点 D 具有轻边 (D, A) , (D, B) , (D, C) , 松弛更新后的最短距离为 $d(A) = 0$, $d(B) = 2$, $d(C) = 3$, $d(D) = 1$, $d(E) = \infty$. 此时 $B[0]$ 桶清空, 结束第一个内循环.
- (4) 集合 S 中具有 A, D 两点, 其重边为 (A, B) , (A, C) , (A, E) , 松弛更新后的最短距离为 $d(A) = 0$, $d(B) = 2$, $d(C) = 3$, $d(D) = 1$, $d(E) = 3$. 此时结束第一个外循环.
- (5) 清空集合 S , 此时最小非空桶为 $B[1]$, 将 B, C, E 三点加入集合 S , 集合

S 具有轻边 (B, C) , (E, B) , 进行松弛操作后距离未发生改变. 此时 $B[0]$ 桶清空, 结束第二个内循环. 由于集合 S 没有重边, 结束第二个外循环.

- (6) 所有桶清空, 算法结束, 输出以 A 为源点的单源最短距离为 $d(A) = 0$, $d(B) = 2$, $d(C) = 3$, $d(D) = 1$, $d(E) = 3$.

总体而言, Δ -Stepping 算法通过引入桶的概念以及优化边的处理, 有效提高了单源最短路径算法在大规模图上的执行效率. 而且本质上该算法涵盖了 Dijkstra 算法和 Bellman-Ford 算法, 当参数 Δ 取值为 1 时, 近似为 Dijkstra 算法; 当参数 Δ 取值为无穷大时, 近似为 Bellman-Ford 算法. 但是 Δ -Stepping 算法对固定的 Δ 值比较敏感, 实验中选择合适的 Δ 对算法的性能影响比较大. 其次, 重复松弛也是影响 Δ -Stepping 算法效率的重要原因之一.

2.4 CUDA 并行框架介绍

随着大数据时代的到来, 处理大规模数据的能力逐渐成为各种算法亟需突破的瓶颈. 依靠单纯的 CPU 吞吐能力已经无法满足日益增长的计算需求. 为了解决图像等领域上的数据处理问题, 图形处理单元 (GPU, Graphics Processing Unit) 应运而生. 在上世纪末, 为了将 GPU 引入到其他领域, NVIDIA 公司发布了 GPU 的第一款概念产品. 随后, 为了充分发挥 GPU 更强大的性能, NVIDIA 推出了 CUDA, 这实质上是基于 C 语言开发的 GPU 编程模型, 专为并行数值计算而设计. CUDA 允许开发者利用 GPU 的并行计算能力, 来加速各种科学计算和数据处理任务, 因此也成为应对大规模数据计算需求的重要工具.

2.4.1 CUDA 软件架构

CUDA 之所以具有强大的并行能力, 主要在于其内核并行框架的卓越性能. CUDA 内核主要由 Grid、Block、Thread 三个部分组成. 一个 Grid 网格包含多个 Block 块, 而每个 Block 块内部则包含按照不同维度组织方式的 Thread 线程构成. 这些维度可以是一维、二维或者三维的. 以一个例子来说明, 图2-10展示了 CUDA 内核架构图, 其中包含一个 2×2 的网格, 每个块内有 3×3 个线程, 最大可并行线程数为 36. CUDA 的这种内核设计允许开发者更灵活地利用并行线程计算资源, 高效地完成各种数据处理任务.

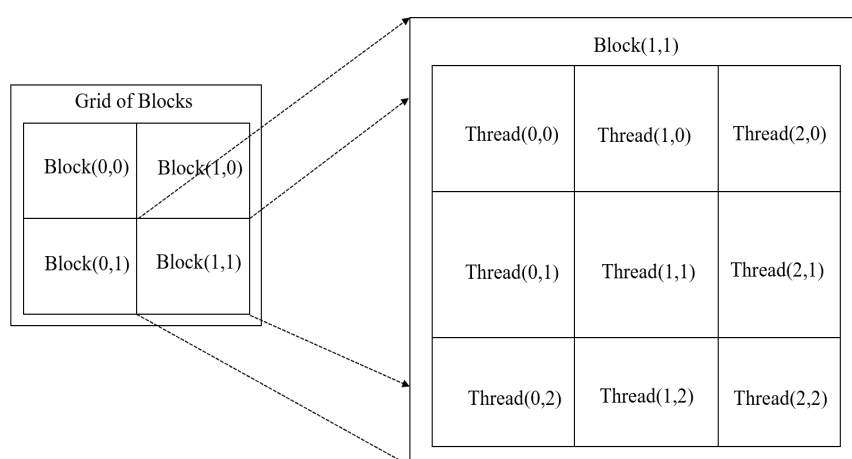


图 2-10 CUDA 软件架构图

2.4.2 GPU 编程模型

GPU 编程模型通常指的是 CPU 与 GPU 之间的协同工作。在硬件上，CPU 与 GPU 通过 PCI Express (PCIe) 总线相连，用于传递指令和数据。一般将 CPU 称为主机端，GPU 称为设备端。在早期，CPU 通常只有一个核，因此无法同时执行多个计算任务，即无法并行处理。如图 2-11 所示，CPU 内部的算术逻辑单元 (ALU) 占比较小，但是 GPU 内部的算术逻辑单元 (ALU) 占比较大，并且具有多个核心，因此 GPU 设备端主要负责执行计算过程简单但数据量庞大的任务。这样 GPU 编程模型巧妙在程序中同时使用 CPU 和 GPU，以充分发挥各自优势，提高整体运行效率。在这种模型下，CPU 主要负责执行程序串行部分或者计算逻辑复杂，GPU 主要执行程序高度并行部分，例如大规模数据并行处理和图形渲染。

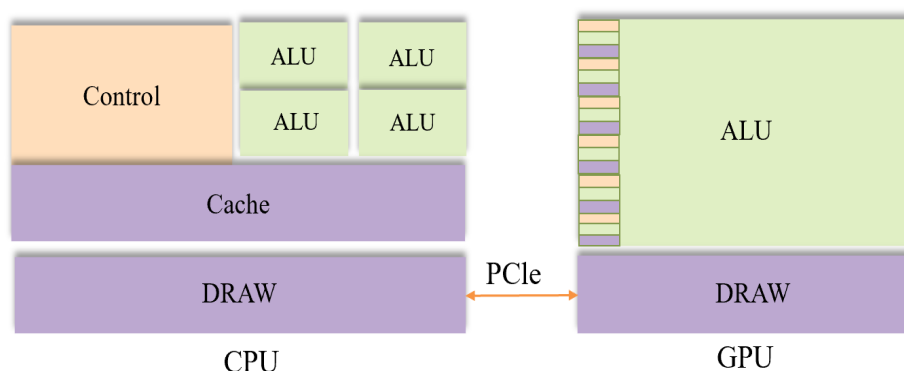


图 2-11 CPU-GPU 硬件图

正如前文所述，由于 GPU 的软件架构与 CPU 不同，因此 CUDA 编程与其基础语言 C/C++ 语言在某些方面略有不同。其中最主要的差异之一是 CUDA 引入了核函数的概念，用于在 GPU 上并行执行计算，方便处理一个或多个线程。CUDA

编程的一般步骤如下:

- (1) 程序拆分: 需要对程序进行拆分, 找出适合由 GPU 并行计算的部分.
- (2) 显存分配: 使用 `cudaMalloc` 函数在 GPU 上分配显存.
- (3) 数据传输: 通过 `cudaMemcpy` 函数将数据从主机内存拷贝到显存.
- (4) 核函数执行: 定义并调用核函数, 核函数会在 GPU 上并行执行, 处理指定数量的线程.
- (5) 结果传回: 使用 `cudaMemcpy` 将计算结果从显存复制回主机内存.

最后, 整个过程中, 还需要负责管理内存和显存的分配与释放, 以确保程序的正确执行. 具体 CUDA 编程模型如图2-12所示.

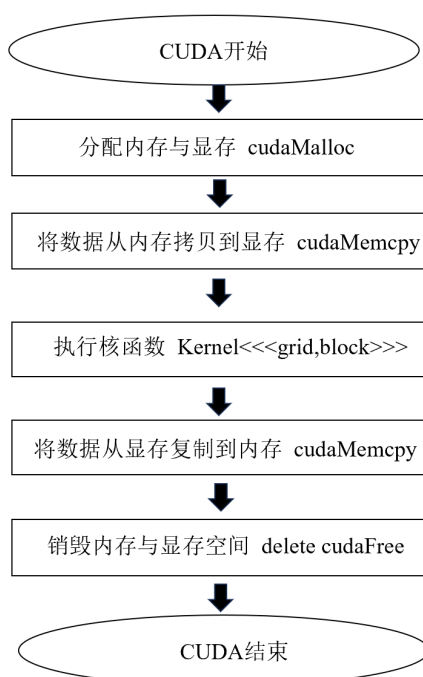


图 2-12 CUDA 编程模型图

2.5 本章小结

本章主要介绍图的相关知识点、三种经典单源最短路径算法原理以及 CUDA 并行框架. 在分析 Dijkstra 算法、Bellman-Ford 算法和 Δ -Stepping 算法的优劣时, 着重从时间复杂度和并行性方面进行探讨. Δ -Stepping 算法的创新之处在于采用分桶处理各顶点, 并对出边进行轻重边分类. 这种创新也体现了选择 Δ -Stepping 算法作为并行改进的优势. 此外, 本章详细介绍了 CUDA 并行框架在处理大规模图时的各方面优势.

第三章 Δ -Stepping 算法改进及并行化介绍

本章首先对 Δ -Stepping 算法与 Dijkstra 算法、Bellman-Ford 算法进行性能分析。接着，从以下三个方面对 Δ -Stepping 算法进行改进：一是针对边的松弛次数进行优化；二是利用最短路径树确定 Δ 的初始值；最后，基于 CUDA 框架并行来加速算法的执行。

3.1 Δ -Stepping 算法性能分析

由第二章分析可知，影响算法性能最主要的指标是：边松弛次数和桶数。

边松弛次数主要影响算法运行时间和临时最短距离更新的有效性，松弛次数过多会无形给算法执行过程增加很多无效松弛。Dijkstra 算法由于每次是从确定的当前最小距离的点开始进行松弛，所以每条边只会松弛一次，松弛总次数为 m 。Bellman-Ford 算法原理基于动态规划，对于每一个顶点都需要进行 $n - 1$ 次松弛操作，所以 Bellman-Ford 算法的松弛次数是最多的。而 Δ -Stepping 算法每次是对当前桶内顶点的出边进行松弛，因为松弛轻边时顶点可能会再次进入桶中，因此 Δ -Stepping 算法的松弛次数介于 Dijkstra 算法和 Bellman-Ford 算法之间。

桶数的选择主要影响算法的并行性能。较小的桶数可能导致并行性能未能充分发挥，而过多的桶数则会引入桶与桶之间的通信，增加额外的时间开销。Dijkstra 算法通过依次选择距离最小的顶点进行松弛操作。在这种情况下，Dijkstra 算法桶的数量可以看作是选取最小距离顶点的个数。Bellman-Ford 算法由于其顶点无差别落入相同桶中，所以其桶个数在任何情况下都是 1。在 Δ -Stepping 算法中，桶数是由 Δ 和所有顶点最短路径的距离最大值 L 确定，即桶数等于 $\frac{L}{\Delta}$ 。由于所有顶点最短路径的初始值未知，所以一般设定 L 为无穷大。在选择合适的 Δ 值下， Δ -Stepping 算法的桶数介于 Dijkstra 算法和 Bellman-Ford 算法之间。这样的选择可以在维持算法的高效性同时，还能适应不同规模图的特性。因此，对于 Δ -Stepping 算法，合理设置桶数不仅可以充分发挥其良好的并行性能，还可以确保算法整体效率的提升。

以下主要从边松弛次数和桶数两方面对 Δ -Stepping 算法进行优化。

3.2 松弛次数优化

Δ -Stepping 算法的效率提升关键在于权衡减少松弛次数. 下文将从边分类优化和剪枝优化两个方面展开讨论.

3.2.1 边分类优化

Δ -Stepping 算法最初引入边的分类是由 Meyer 和 Sanders^[17] 在 2003 年的工作中首次提出. 其中, 参数 Δ 用于确定轻重边的分类标准: 当边权值 $w(u, v)$ 小于等于 Δ 时, 称其为轻边; 当边权值 $w(u, v)$ 大于 Δ 时, 则为重边. 这种分类的目的在于, 在清空当前桶时, 对桶内顶点的出边进行巧妙判断. 具体而言, 轻边由于权值相对较小, 在松弛阶段容易导致出边顶点落入当前桶, 触发再次清桶的操作. 与此不同, 重边由于其边权值大于 Δ , 即使在满足松弛阶段条件下更新其出边顶点的距离, 结果仍使得出边顶点属于后向桶, 而不会再次回到当前桶. 因此, 在清空当前桶时松弛重边是多余的.

所以算法过程主体被分为两个阶段: 第一阶段, 先处理当前桶内顶点发出的所有轻边, 当所有顶点都稳定下来不再落入当前桶时该阶段结束; 第二阶段, 处理当前桶内顶点发出的所有重边. 由于轻边和重边在算法中对于距离更新起到不同的作用, 且第一阶段主要考虑顶点是否会重复落入当前桶, 因此进行有效的边分类有助于减少松弛次数.

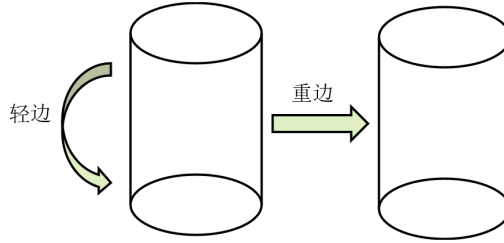


图 3-1 边分类优化图

仔细分析可发现, 并非所有边权值 $w(u, v)$ 小于等于 Δ 的出边顶点都会触发再次清桶的操作. 通过进一步分类, 当出边 (u, v) 满足 $d(u) + w(u, v) \leq (i + 1)\Delta - 1$ 时, 出边顶点会再次落入当前桶; 而出边 (u, v) 满足 $d(u) + w(u, v) > (i + 1)\Delta$ 时, 顶点会落入后向桶中. 其实通过观察整个最短路径求解过程可以发现, 权值相对更小的边更容易构成最短路径, 所以轻边对最短距离的更新起到更为明显的作用. 因此对轻重边的重新分类可以突出有效轻边从而减少边重复松弛次数, 并更明确地区分 Δ -Stepping 算法内外循环的作用. 具体示意图如图3-1. 优化后的算法简称为 EDS (Edge Δ -Stepping) 伪代码如3.1所示.

算法 3.1 Δ -Stepping 边分类优化算法

```

1: Input: Graph  $G = (V, E, W)$ , source vertex  $s$ 
2: Output: Shortest distances  $d(v)$  for all  $v \in V$ 
3: Initialization:
4:  $\text{relax}(s, 0)$ 
5:  $i \leftarrow 0$ 
6: while  $B \neq \emptyset$  do
7:    $S \leftarrow \emptyset$ 
8:   for each  $v \in V$  do
9:      $\text{heavy}(v) \leftarrow \{(u, v) \in E : d(u) + w(u, v) > (i + 1)\Delta\}$ 
10:     $\text{light}(v) \leftarrow \{(u, v) \in E : d(u) + w(u, v) \leq (i + 1)\Delta - 1\}$ 
11:     $d(v) \leftarrow \infty$ 
12:   end for
13:   while  $B[i] \neq \emptyset$  do
14:     (b1) Add To Request:
15:      $\text{Req} \leftarrow \{(w, d(u) + w(u, v)) : u \in B[i] \wedge (u, v) \in \text{light}(u)\}$ 
16:      $S \leftarrow SUB[i]$ 
17:      $B[i] \leftarrow \emptyset$ 
18:     (c1) Relax:
19:     for each  $(u, x) \in \text{Req}$  do
20:        $\text{relax}(u, v)$ 
21:     end for
22:   end while
23:   (b2) Add To Request:
24:    $\text{Req} \leftarrow \{(w, d(u) + w(u, v)) : v \in S \wedge (u, v) \in \text{heavy}(u)\}$ 
25:   (c2) Relax:
26:   for each  $(u, x) \in \text{Req}$  do
27:      $\text{relax}(u, v)$ 
28:   end for
29:    $i \leftarrow i + 1$ 
30: end while

```

3.2.2 剪枝优化

Δ -Stepping 算法的优化在于如何避免无效松弛，以快速找到最短路径. 从最终确定最短路径的有效松弛角度往回看，通过定位具有特殊性质的点和边，可以减少无效松弛，从而更快地找到最短路径. 图3-2展示具有特殊性质的点和边的情况：

- (1) 无效边：对于轻边而言，如果发现顶点 v 属于 $B[i]$ 之前的桶，则无需将边 (u, v) 加入松弛列表. 例如，在图3-2(a)的示例中，虽然边 (A, E) 和 (A, B) 都满足轻边的条件，但是显然 $d(A) + w(A, E) > d(E)$ ，因此边 (A, E) 无需被纳入轻边松弛列表. 对于重边来说，如果发现顶点 v 属于当前桶或者 $B[i]$ 之前的桶，边 (u, v) 也无需被纳入请求列表. 这是因为在这种情况下，即使将边 (u, v) 加入重边松弛列表，顶点 v 的最短距离不会发生改变. 从图3-2(b)可以看出，虽然边 (A, D) 和 (B, E) 都属于重边，但是由于顶点 E 属于 $B[i]$ 之前的桶，所以 $d(B) + w(B, E) > d(E)$ ，因此对边 (B, E) 进行松弛是无效的. 由于这些边并不会对最终结果产生影响，所以将它们加入请求列表是没有意义的.

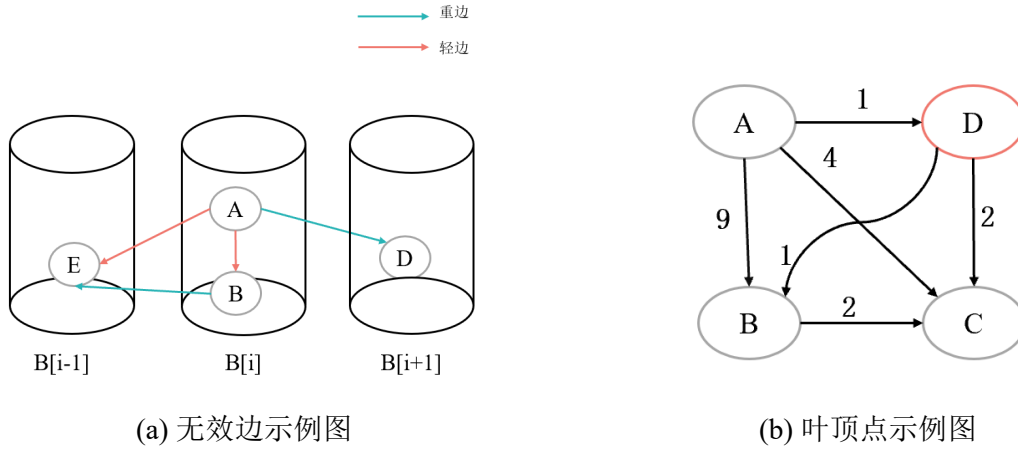


图 3-2 剪枝优化图

(2) 叶顶点：叶顶点是指入度为 1 的顶点，因为其只有一条边指向它。如图 3-2(b) 所示，顶点 D 是一个叶顶点，因为只有边 (A, D) 可到达它。对于这样的叶顶点 D ，其距离更新仅能通过边 (A, D) 进行。如果观察到边 (A, D) 已经被加入待松弛列表，可以直接更新 $d(D)$ 为 $d(A) + w(A, D)$ ，无需再进行 $d(D) < d(A) + w(A, D)$ 这一条件的判断。因此有效识别叶顶点，可以减少松弛条件判断计算，提高算法效率。

算法 3.2 Δ -Stepping 剪枝优化阶段

```

1: while  $B[i] \neq \emptyset$  do
2:   (b1) Add To Request:
3:    $Req \leftarrow \{(u, d(u) + w(u, v)) : u \in B[i] \wedge (u, v) \in light(u) \wedge v \in B[j], j \geq i\}$ 
4:    $S \leftarrow SUB[i]$ 
5:    $B[i] \leftarrow \emptyset$ 
6:   (c1) Relax:
7:   for each  $(u, x) \in Req$  do
8:     if  $v = \text{leaf-nodes}$  then
9:       update  $v$ 
10:    else
11:      relax( $u, v$ )
12:    end if
13:  end for
14: end while
15: (b2) Add To Request:
16:  $Req \leftarrow \{(u, d(u) + w(u, v)) : v \in S \wedge (u, v) \in heavy(u) \wedge v \in B[j], j > i\}$ 
17: (c2) Relax:
18: for each  $(u, x) \in Req$  do
19:   if  $v = \text{leaf-nodes}$  then
20:     update  $v$ 
21:   else
22:     relax( $u, v$ )
23:   end if
24: end for
    
```

因此, 剪枝优化算法采用对点和边进行不同标记的优化策略, 剔除了请求列表中的无效边, 以及减少松弛操作直接进行距离更新. 这一策略从最终确定最短路径的角度出发, 充分利用图的特性, 减少无效请求列表的冗余元素和松弛条件的计算次数以提高算法的效率. 该算法简称为 PDS (Pruning and Edge Δ -Stepping) 算法, 其伪代码具体如3.2所示.

3.3 基于最短路径树确定 Δ 值

在 Δ -Stepping 算法中, Δ 值的设定对算法性能起到关键作用. 一方面, Δ 值的选择影响轻重边的划分. 由于重边的权值较大, 它不会在松弛过程中再次落入当前桶, 因此 Δ 值主要影响轻边是否在当前桶内再次松弛, 从而优化松弛次数. 另一方面, Δ 值直接影响桶的数量, 进而影响算法的并行效果. 较大的 Δ 值导致桶变宽, 减少桶的数量, 增加每个桶内的顶点数量, 提高桶内的并行性. 相反, 较小的 Δ 值导致桶变窄, 增加桶的数量, 减少每个桶内的顶点数量, 降低桶内的并行性. 由于不同图的特性差异很大, 直接找到通用的 Δ 值并不现实. 因此, 建立图的特性与 Δ 值之间的关联关系可能是寻找通用函数的关键.

3.3.1 最短路径树

在图论中, 最短路径树 (SPT) 是一种数据结构, 揭示图中从源点到其他所有顶点的最短路径. 它的形成过程是通过不断更新根顶点 (即源点) 的最短距离和最短路径上的前驱顶点, 从而最终形成一颗树状结构. 所以最短路径树反映了根顶点到所有顶点的步数和总距离这一关系, 其中步数指的是源点到目标顶点的最短路径上所经过的顶点数, 总距离则是指这条最短路径上所有边的权重之和. 最短路径树的生成不仅提供了最短路径的具体信息, 还为图的结构和顶点之间的最优路径布局提供了重要见解. 图3-3是最短路径树的简单示例:

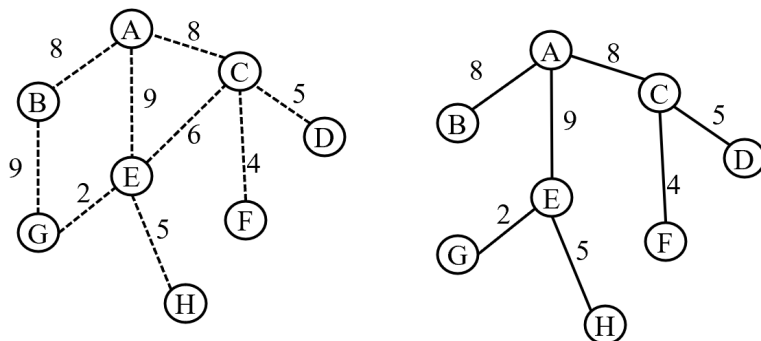


图 3-3 最短路径树示例图

3.3.2 基于 SPT 确定 Δ 值

图的特性可以考虑包括权重和度数等因素. 在先前的研究中, 一些学者提出了基于最短路径树优化单源最短路径算法. 张钟^[52] 提出了基于最短路径树优化路径初始值的方法, 陈钧吾^[53] 提出了基于最短路径树估算图中顶点之间的最短距离从而减少松弛数量和优化顶点处理顺序改进点. 可以发现, 最短路径树包含图的明显特性, 借助最短路径树可以探究单位深度下的平均距离, 即一步所反映的距离长度. 如果平均距离较长, 那么在相同深度下可能需要更大的 Δ 值, 以便更好地覆盖长距离路径; 反之, 如果平均距离较短, 可能需要较小的 Δ 值, 以便更精细地探索短距离路径. 所以最短路径树的平均距离与平均深度的商值作为 Δ 值可以有效平衡长距离最短路径和短距离最短路径的探索, 有利于在算法的初期阶段快速收敛, 因为其可以更好地覆盖图中的各个部分, 而不是过于偏重于某一特定区域. 本文选择了基于 Bellman-Ford 算法构建最短路径树的方法. 这是因为 Bellman-Ford 算法能够直接生成最短路径树, 这样便于利用最短路径树的平均距离与平均深度之比来计算 Δ 的初始值. 其伪代码如3.3所示.

算法 3.3 基于 Bellman-Ford 算法计算 Δ 初始值

```

1: Input: Graph  $G = (V, E, W)$ , source vertex  $s$ 
2: Output: Shortest distances  $d(v)$  for all  $v \in V$ 
3: Initialization:
4: for each  $v \in V$  do
5:    $d(v) \leftarrow \infty$ 
6:    $depth(v) \leftarrow 0$ 
7: end for
8:  $d(s) \leftarrow 0$ 
9: for  $i \leftarrow 1$  to  $|V| - 1$  do
10:   for all edges  $(u, v) \in E$  do
11:     if  $d(u) + w(u, v) < d(v)$  then
12:        $d(v) \leftarrow d(u) + w(u, v)$ 
13:        $depth(v) \leftarrow depth(u) + 1$ 
14:     end if
15:   end for
16: end for
17:  $avg\_dist = avg(d)$ 
18:  $avg\_depth = avg(depth)$ 
19:  $\Delta = avg\_dist / avg\_depth$ 
20: return  $\Delta$ 

```

3.4 算法的并行化

在单源最短路径算法中, 并行策略的选择对于算法的性能至关重要, 常见策略有以下三种: 基于顶点分配的并行化、基于邻接矩阵划分的并行化和基于边分配的并行化. 在 Δ -Stepping 算法的求解过程中, 可以观察到当前桶中顶点添加到请求

列表和松弛轻重边的两个关键阶段都具有并行化的可能. 基于这两个阶段的特点, 可以采用混合并行方法来优化 Δ -Stepping 算法. 具体而言, 清桶阶段可以采用顶点并行的方式进行处理, 以充分利用顶点间的独立性. 而在松弛阶段, 可以采用边分配的并行策略, 以便高效地处理边之间的关联. 通过这种混合并行的优化方式, 可以更好地平衡并行计算负载, 提高算法的整体性能.

3.4.1 清桶阶段顶点并行

在 Δ -Stepping 算法的内循环阶段中, 当需要清空当前桶内的顶点时, 必须将所有这些顶点添加到请求列表中. 由于同一桶内的各顶点之间没有先后顺序的依赖关系, 因此可以采用顶点并行策略来对这一阶段进行优化处理. 顶点并行的处理方式是每个顶点均匀地分配给不同的线程, 然后在每个线程上执行标记顶点和将顶点加入请求列表等操作. 这种优化策略的核心思想是充分利用桶内顶点间的独立性, 通过并行处理每个顶点, 从而提高算法的整体效率. 采用顶点并行的方式, 可以在同一时间处理多个顶点的清桶操作, 有效减少了处理请求列表的总时间.

3.4.2 松弛阶段边分配并行

在 Δ -Stepping 算法的松弛阶段, 考虑到可以同时松弛当前桶中所有顶点的轻边或者重边, 因此选择采用边分配并行策略对这一阶段进行优化处理. 与传统的 Δ -Stepping 串行算法逐个处理所有轻边或者重边不同, 边分配并行策略将待松弛的边均匀分配给不同的线程, 每个线程负责执行松弛和更新顶点桶编号的操作. 这种并行策略的优势在于能够更充分地利用 GPU 等并行计算设备的性能, 同时提高算法的执行效率. 通过同时处理多个边, 边分配并行策略有效地减少了串行算法中边的处理顺序对性能的影响. 因此, 这种优化对于加速 Δ -Stepping 算法在大规模图上的处理能力具有重要作用.

通过这种混合并行的设计, Δ -Stepping 算法充分考虑了顶点并行和边分配并行在清桶和松弛阶段的优势, 进而在 GPU 硬件上发挥最大的并行处理能力, 提高算法在处理大规模图时的性能表现. 在 Δ -Stepping 算法并行优化中, 除在算法设计上巧妙地结合顶点并行和边分配并行的混合模式, 也考虑在并行计算模型上采用融合线程块和线程的并行执行策略. 在处理较小规模图时, 单独利用线程并行可能对性能影响不大. 然而, 当面对较大规模的图时, 纯粹依赖线程并行可能面临多方面挑战, 其中之一是算法求解所需时间较长, 导致 GPU 利用率不尽如人意. 在这种情况下, 需要综合考虑并采用其他优化策略, 以充分发挥 GPU 的并行处理潜力. 通过同时利用线程块和线程的并行性, 能够更充分地发挥 GPU 强大的并行处理能力, 从而优化算法在大规模图上的执行效率. 具体混合并行算法设计如3.4所示.

算法 3.4 Δ -Stepping with Hybrid Parallelism

```

1: while  $B \neq \emptyset$  do
2:    $S \leftarrow \emptyset$ 
3:   while  $B[i] \neq \emptyset$  do
4:     (b1) Add To Request (Vertex-Parallel):
5:     Parallel Add To Request( $B[i]$ ,  $S$ ,  $Req$ )
6:     (c1) Relax (Edge-Parallel):
7:     Parallel Relax( $Req$ )
8:   end while
9:   (b2) Add To Request (Vertex-Parallel):
10:  Parallel Add To Request( $S$ ,  $Req$ )
11:  (c2) Relax (Edge-Parallel):
12:  Parallel Relax( $Req$ )
13:   $i \leftarrow i + 1$ 
14: end while
15: cudaMemcpy  $d$  from GPU to CPU

```

3.5 本章小节

本章通过对经典单源最短路径算法的性能进行分析, 发现 Δ -Stepping 算法运行时间可以提升的地方, 并找到减少松弛数量和桶数之间进行权衡的关键点.

首先, 基于边重复落入当前桶的标准对轻重边进行划分, 有效减少轻边松弛次数. 进一步分析发现, Δ -Stepping 算法在对边进行松弛时存在较多无效松弛的情况, 因此本文采用对当前桶的边进行无效边判断并剔除的方法. 同时, 也发现入度为 1 的叶顶点可以直接进行距离更新, 无需进行额外的松弛条件判断. 其次, 通过利用图的特性, 提出了一种基于最短路径树的距离和深度启发式方法来确定 Δ 值, 该值不仅影响轻重边的分类, 也对后续算法的并行实现产生一定影响. 最后, 通过分析 CUDA 并行框架和算法各个阶段的特点, 采用顶点分配并行和边分配并行的混合并行策略. 这样的混合并行设计不仅考虑到算法本身的并行性, 同时也充分考虑了 GPU 硬件架构的特点, 为在不同规模的图上实现高效的并行计算提供了更加灵活的选择.

第四章 数值实验与结果分析

本章旨在对第三章提出的优化算法进行实验分析,以验证其有效性.首先,介绍实验环境、实验数据和评价指标,以确保实验的准确性和可靠性.其次,在运行时间、边松弛次数以及桶的个数三个维度上对比分析 Δ -Stepping 算法、Dijkstra 算法和 Bellman-Ford 算法.然后,将优化后的 Δ -Stepping 算法与原始算法进行对比.最后,在真实图数据上测试优化算法,以验证其实际表现.

4.1 实验准备

本节将介绍涉及的实验环境,所需数据集和实验结果评价指标.

4.1.1 实验环境

本章实验所涉及的软硬件配置如下表4-1,实验中所有算法基于 C++ 实现,通过 CUDA 框架实现并行. CUDA 并行实现过程中单个线程块线程数设置为 1024,网格数量依据 CUDA 设备属性和最大并行任务量进行计算.

表 4-1 软硬件环境

项目	参数
CPU	Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz 40 核
CPU Memory	128G
GPU	NVIDIA GeForce RTX 3090
GPU Memory	24G
OS	Linux Ubuntu 18.04.5 LTS
CUDA Version	CUDA V12.4.99

4.1.2 数据集介绍

本章的实验数据一部分来源于使用 Python3.9 中的 NetworkX 库生成的常见网络图.这些数据在第二章已经介绍过其数据集的特点.主要选择 BA 无标度网络图和 ER 随机网络图作为实验数据的一部分.选择 BA 无标度网络图的原因在于其顶点度数分布的独特性,即少数顶点拥有相对较高的度数,而大部分顶点度数较低.

这种特性使得 BA 无标度网络图能更真实地反映现实世界中众多网络的结构特征, 如社交网络、互联网等. 相比之下, 选择 ER 随机网络图则是出于其连接随机性的考虑. 这种随机性使得 ER 随机网络图成为一个理想的对比基准, 有助于深入理解优化算法在不同网络结构下的性能表现.

另一部分是真实图数据, 来自 KONECT^[54] 网站. 一共选取四幅图, 其中稀疏图是 twitter 数据集和 flickr-links 数据集, 稠密图是 wikipedia-links 数据集和 livemocha 数据集. twitter 数据集记录的是 twitter 上各用户之间关注的信息, 其中顶点是用户, 边指的是各用户之间的关注关系. flickr-links 数据集和 livemocha 数据集记录的是平台上用户社交网络网络信息, 其中顶点是用户, 边是用户社交关系. wikipedia-links 数据集记录的是维基链接之间的连接关系, 其中顶点是维基百科条目, 边指的是维基条目链接.

4.1.3 评价指标

为了更好地观察到优化算法的效果提升, 本文采用加速率刻画程序优化效果. 加速率公式如下:

$$R = \frac{T_0 - T_1}{T_0},$$

其中 T_0 是程序优化前的运行时间, T_1 是程序优化后的运行时间. 从公式可以直观看到, 加速率越大, 说明优化程序前后带来的时间开销差值占原程序的比值变大, 反应优化效果越好.

4.2 经典算法对比分析

在第二章中提到, 单源最短路径算法涵盖了 Δ -Stepping 算法、Dijkstra 算法和 Bellman-Ford 这三种经典算法. 在进行 Δ -Stepping 优化性能分析之前, 有必要先对这三种经典单源最短路径算法进行性能对比, 以确定可能存在的性能瓶颈. 本节选取了 BA 无标度网络图作为实验数据集, 由于网络生成图是无权图, 为了实验准确性, 本节对图中每条边随机赋予了 $[1, 100]$ 之间的权重, 以比较算法之间的性能差异. 此外, 为了全面评估各算法在不同类型图上的性能表现, 还对具有相同顶点数的稀疏和稠密 BA 无标度网络图进行了实验. 为减少实验误差, 本节实验结果均基于五次重复实验计算得出平均值. 接下来从松弛次数, 桶个数和算法运行时间三个角度分析不同算法的性能差异.

从图4-1和图4-2中的边松弛次数对比图可以观察到, 随着图的规模增大, 边松弛次数也在增加, 特别是在稠密图中, 边松弛次数增长迅速. 无论是在稀疏图还是稠密图中, Dijkstra 算法的边松弛次数最少, Δ -Stepping 算法次之, 而 Bellman-Ford

算法的边松弛次数最多, 这与第三章的性能分析结论相一致. 对 Δ -Stepping 算法的边松弛次数来说, 可以看到在相同规模的图中, 随着 Δ 值从 1 逐渐增加到 100, 边松弛次数开始时变化较缓慢, 但是当 Δ 值增大到一定程度之后, 边松弛次数开始迅速增长.

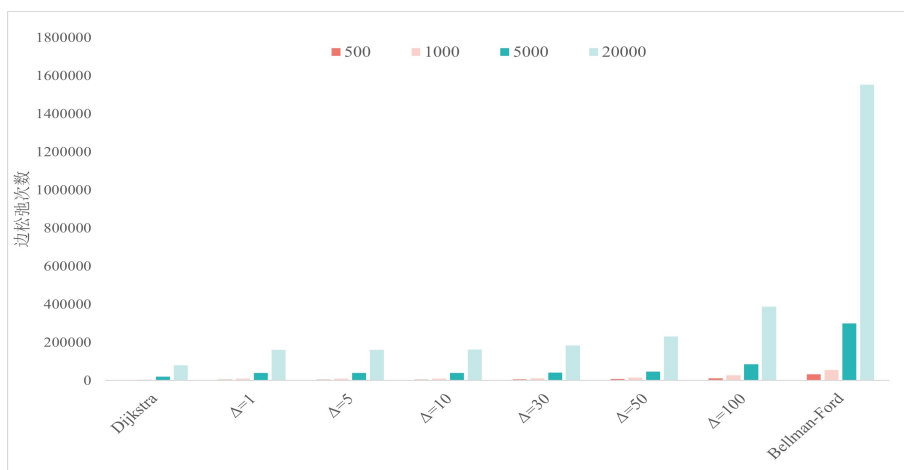


图 4-1 稀疏图边松弛次数对比

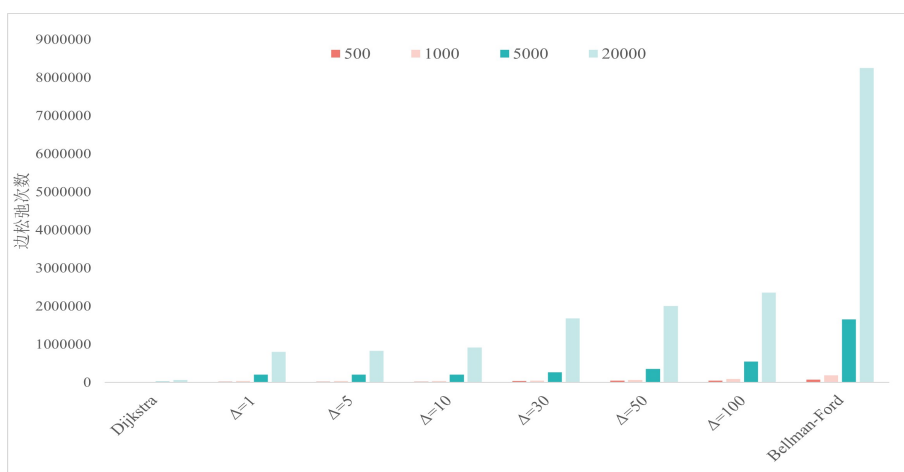


图 4-2 稠密图边松弛次数对比

在图4-3和图4-4中, 由于不同算法的桶个数差异显著, 故采用对桶个数取对数形式进行比较. 为便于比较, 图中将桶个数为 1 (即对数结果为 0) 的情况显示为 1. 从图中可以观察到, Dijkstra 算法的桶个数数量最多. 这是因为 Dijkstra 算法中桶个数的定义是从临时距离数组中取出最短距离顶点的总个数, 相当于 Δ -Stepping 中 Δ 取 1 的连续情况. 相比之下, Bellman-Ford 算法在任何情况下桶个数都是 1, 这是由于其顶点无差别落入相同桶中, 所以其桶个数在任何情况下都是 1. 此外, 当 Δ -Stepping 算法中 Δ 值增大到一定程度时, 所有顶点都会落在一个桶中. 因此, Δ -Stepping 算法的桶个数介于两种算法之间, 而且这个数值可以通过 Δ 进行调整,

这一特点也使得 Δ -Stepping 算法更适合并行化实现。

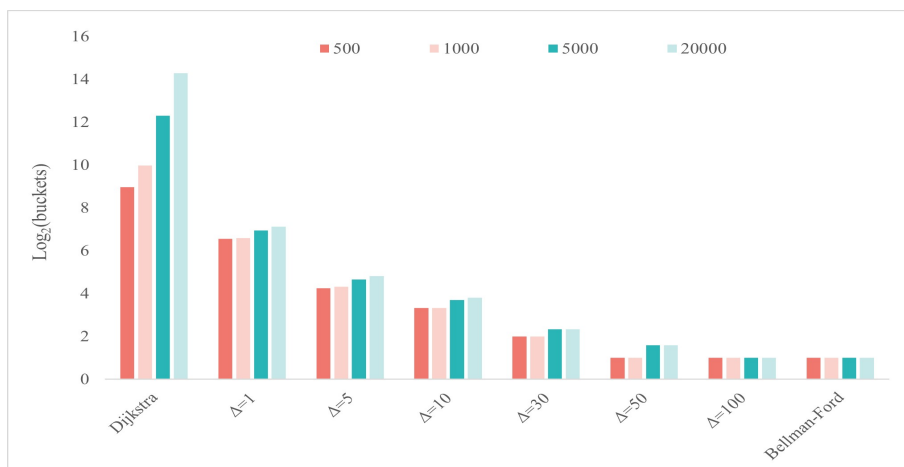


图 4-3 稀疏图桶个数对比



图 4-4 稠密图桶个数对比

最后对三种算法在不同图的运行时间进行对比，表4-2和表4-3分别展示了三种算法在稀疏图和稠密图上的运行时间。可以发现，在图的顶点数较少时，三种算法的运行时间大致相当。然而，随着顶点数的增加， Δ -Stepping 算法的运行时间呈现平稳增长趋势，而 Dijkstra 算法和 Bellman-Ford 算法的运行时间开始呈指数增长趋势。特别地，当顶点数从 500 增加到 20000 时，Bellman-Ford 算法的运行时间增长超过 100 倍，这表明 Bellman-Ford 算法不适用于大规模图。从稀疏图和稠密图的角度来看， Δ -Stepping 算法的运行时间明显优于另外两种算法。在稠密图上， Δ -Stepping 算法整体的运行时间开销仅有 Dijkstra 算法的 10%，Bellman-Ford 算法的 4%。

结合三种算法运行时间、边松弛次数和桶个数来看， Δ -Stepping 算法不仅在边松弛次数和桶个数处于比较适中的水平，而且在不同规模图中的运行时间表现也比较好。所以对 Δ -Stepping 原始算法进行深入研究改进显得尤为重要。但是从以

上分析来看, 提升 Δ -Stepping 算法性能关键在于如何减少边松弛次数和选取合适的参数 Δ 值.

表 4-2 稀疏图经典算法运行时间对比 (单位: 秒)

Vertices	Edges	Dijkstra	Bellman-Ford	Δ -Stepping					
				$\Delta=1$	$\Delta=5$	$\Delta=10$	$\Delta=30$	$\Delta=50$	$\Delta=100$
500	2475	0.01	0.02	0.00	0.01	0.00	0.00	0.01	0.01
1000	4975	0.03	0.09	0.01	0.01	0.01	0.01	0.02	0.03
5000	19984	0.62	1.51	0.05	0.07	0.07	0.07	0.08	0.15
20000	79984	10.26	24.55	0.57	0.65	0.69	0.76	0.97	1.61

表 4-3 稠密图经典算法运行时间对比 (单位: 秒)

Vertices	Edges	Dijkstra	Bellman-Ford	Δ -Stepping					
				$\Delta=1$	$\Delta=5$	$\Delta=10$	$\Delta=30$	$\Delta=50$	$\Delta=100$
500	9600	0.02	0.08	0.01	0.01	0.01	0.02	0.02	0.03
1000	19600	0.08	0.27	0.03	0.03	0.03	0.03	0.05	0.07
5000	99600	1.72	6.38	0.12	0.17	0.19	0.36	0.47	0.56
20000	399600	27.98	102.27	0.89	1.15	1.26	2.50	3.74	5.35

4.3 松弛优化算法效果分析

本节首先对松弛算法进行不同优化策略实验分析, 然后再对叠加优化策略进行综合实验分析. 实验数据集选取顶点数依次为 500, 1000, 5000, 20000 的 BA 无标度网络数据集和 ER 随机网络数据集, 并在相同顶点数下设置稀疏图和稠密图. 因 Δ 参数设置对实验效果影响比较大, 所以设置 Δ 参数组为 5、10、20、30、50、100, 以对比不同参数下的优化效果. 鉴于松弛算法主要是通过减少边松弛次数从而影响算法整体运行时间, 所以本节也会采取边松弛减少率这一指标对松弛算法优化效果进行综合评价. 为减少实验误差, 本节实验结果均基于五次重复实验计算得出平均值.

4.3.1 EDS 实验分析

图4-5和图4-6分别展示了 EDS 算法在 BA 无标度网络图和 ER 随机网络图下, 不同 Δ 值对应的加速率效果. 实验结果表明, EDS 算法在多数情况下表现出良好的加速性能, 其加速率普遍大于 0. 在同一实验条件下, 不同 Δ 取值对 EDS 算法的表现效果产生显著影响. 当 Δ 取值适宜时, EDS 算法可达到较高的加速率, 最高可达 37%. 然而, 对于稠密图而言, 当 Δ 值过大导致桶的宽度过宽时, 加速率会出现下降趋势. 这是因为此时顶点主要集中在少数几个桶中, EDS 算法可能将大部分边划分为轻边, 从而导致算法性能下降. 此外, 值得注意的是, 实验中出现了加速率小于 0 的情况, 如在 BA 无标度网络图顶点数为 500 的稀疏图和稠密图中. 这主要是因为在这些情况下, 边的重新定义和分类所带来的计算开销超过了算法本身减少边松弛次数所带来的性能提升, 导致算法运行时间相对于原始算法有所增加.

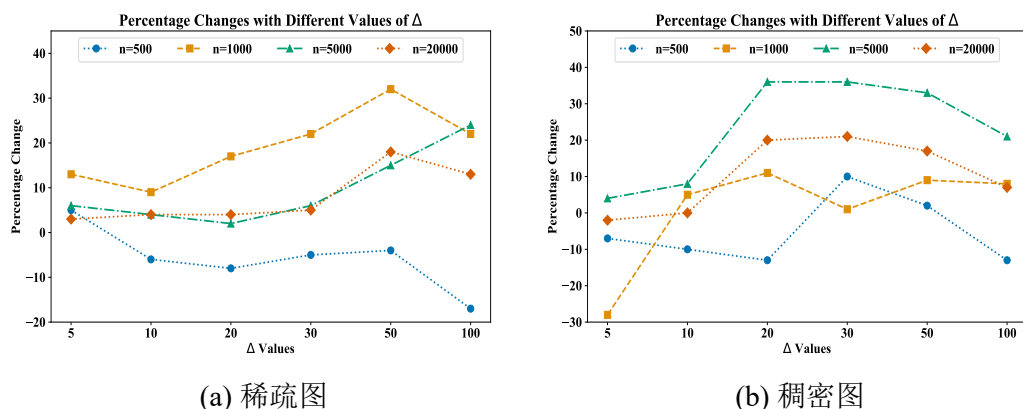


图 4-5 EDS 算法在 BA 无标度网络图上的加速率

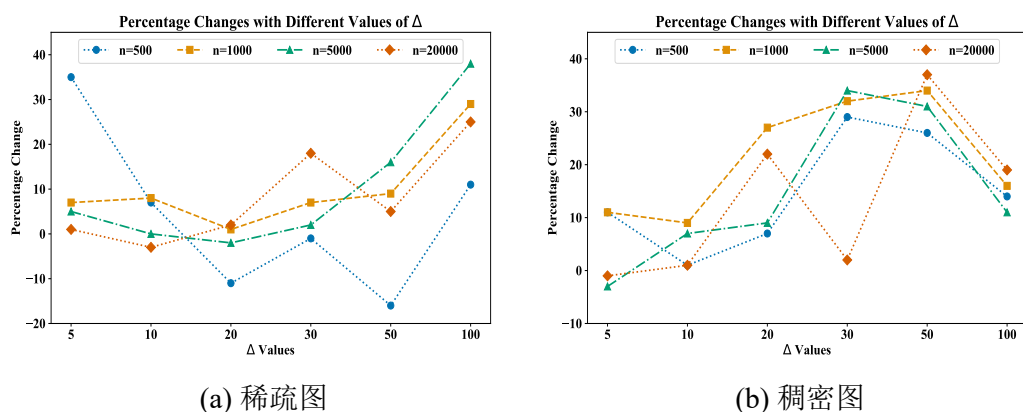


图 4-6 EDS 算法在 ER 随机网络图上的加速率

4.3.2 PDS 实验分析

通过对比图4-7和图4-8中不同图下的加速率折线图,可以观察到 PDS 算法在多数情况下均实现了正加速,整体平均加速率达到 28%,这表明反应剪枝优化对于提升算法效率、缩短运行时间具有显著作用.无论是 BA 无标度网络图还是 ER 随机网络图,PDS 算法加速率折线图的走势都呈现出较为一致的趋势.值得注意的是,PDS 算法在稠密图上的性能表现优于稀疏图,其平均加速率相较于稀疏图高出 16%,这进一步验证了 PDS 算法在不同类型图结构中的有效性.

关于 Δ 取值的影响,实验结果表明,在 Δ 取值适中的情况下,PDS 算法的加速率表现最佳,最高可达 67%.然而,当 Δ 值增大到一定程度后,各规模图形的加速率均开始呈现下降趋势,尤其在稠密图中下降更为明显.这是由于较大的 Δ 值导致顶点过于集中在少数几个桶中,影响了 PDS 算法对无效边的准确判断,从而增加了边松弛次数.因此,在实际应用中,需要根据图的特性和需求来选择合适的 Δ 值,以发挥 PDS 算法的最佳优势.

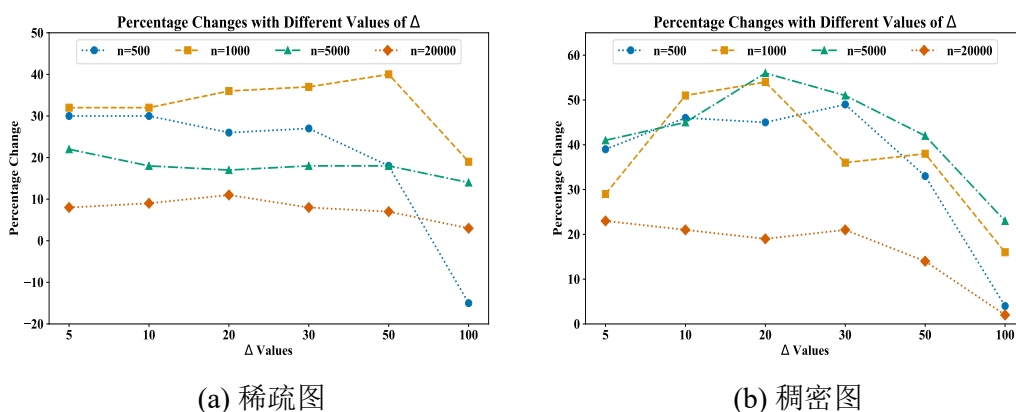


图 4-7 PDS 算法在 BA 无标度网络图上的加速率

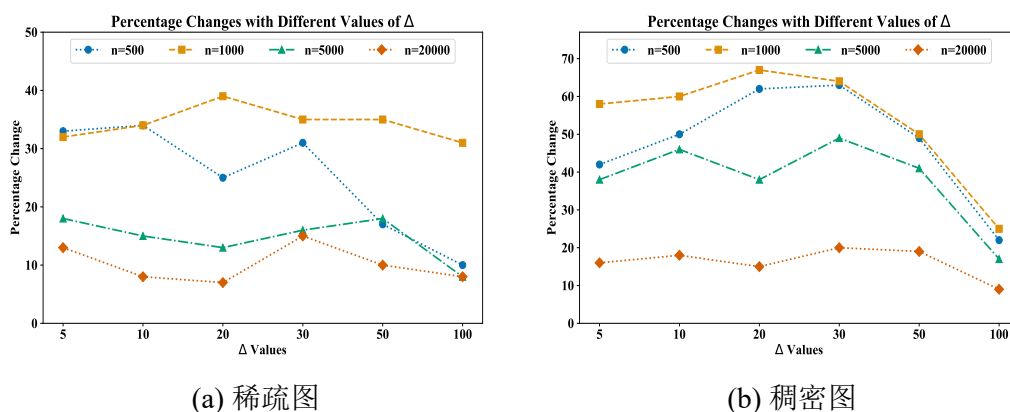


图 4-8 PDS 算法在 ER 随机网络图上的加速率

4.3.3 综合实验分析

由于 EDS 算法和 PDS 算法都是对算法松弛次数进行优化, 因此将这两类优化策略叠加使用并进行综合分析十分重要. 图4-9和图4-10分别是对松弛算法在 BA 无标度网络图和 ER 随机网络图下的效果分析.

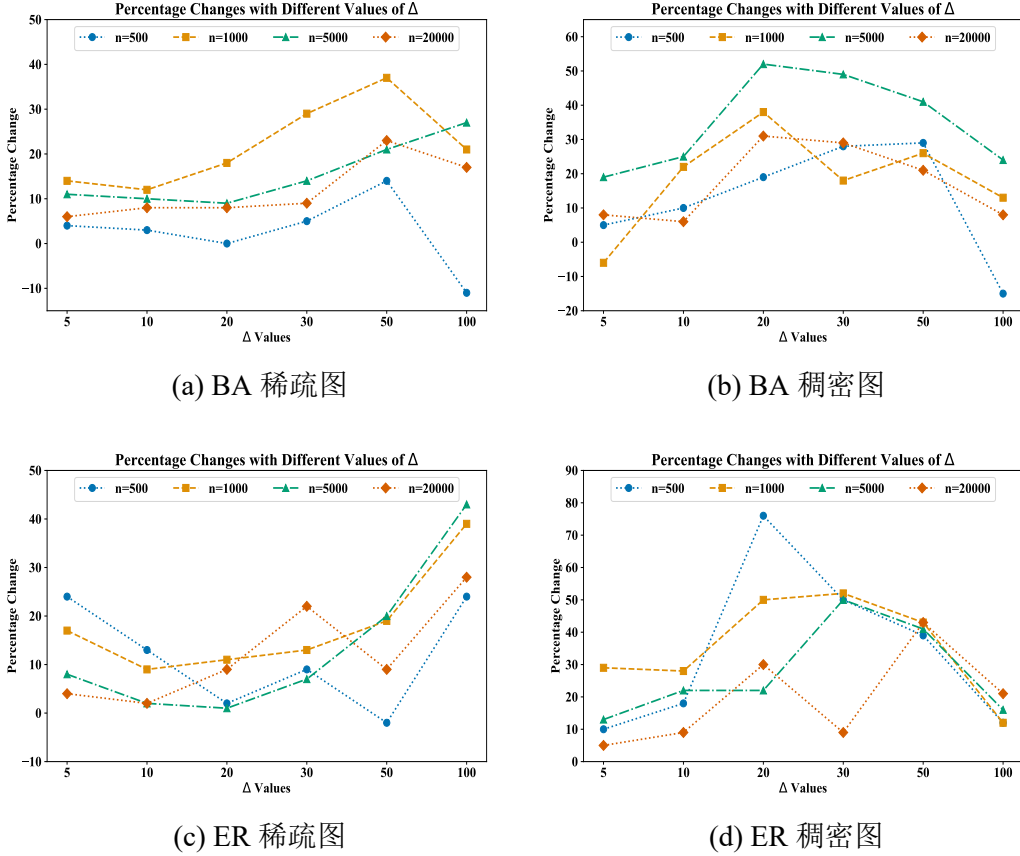


图 4-9 松弛算法在各网络图上的加速率

从加速率的角度来看, 松弛算法的两种叠加优化策略在稠密图上的优化效果总体表现优于稀疏图, 这一规律在各规模图上均得到了体现, 且加速率的走向基本保持一致. 然而, 值得注意的是, 在某些情况下仍会出现负优化的情况, 即加速率小于 0. 这表明在这些特定情境下, 松弛算法虽然减少了边松弛次数, 但其所带来的计算负担却超过了这一优化所带来的收益. 不过, 在 Δ 取值适中的情况下, 松弛算法的表现效果较为理想, 其加速率最高可达 76%, 充分展现了叠加优化策略在提升算法性能方面的潜力.

进一步分析边松弛减少率, 可以发现稠密图上的边松弛减少效果明显优于稀疏图. 这是因为稠密图中顶点之间的连接更为紧密, 边的数量更多, 从而提供了更多的优化空间. 松弛算法在稠密图上能够更有效地识别并减少不必要的边松弛操

作, 从而提高算法的效率. 此外, 在不同 Δ 取值下, 稠密图的边松弛减少率基本保持在 60% 以上. 值得注意的是, 当 Δ 取值较小时, 边松弛减少率也相对较低. 这是因为较小的 Δ 值导致算法对边的分类不够准确, 从而降低边松弛减少的效果.

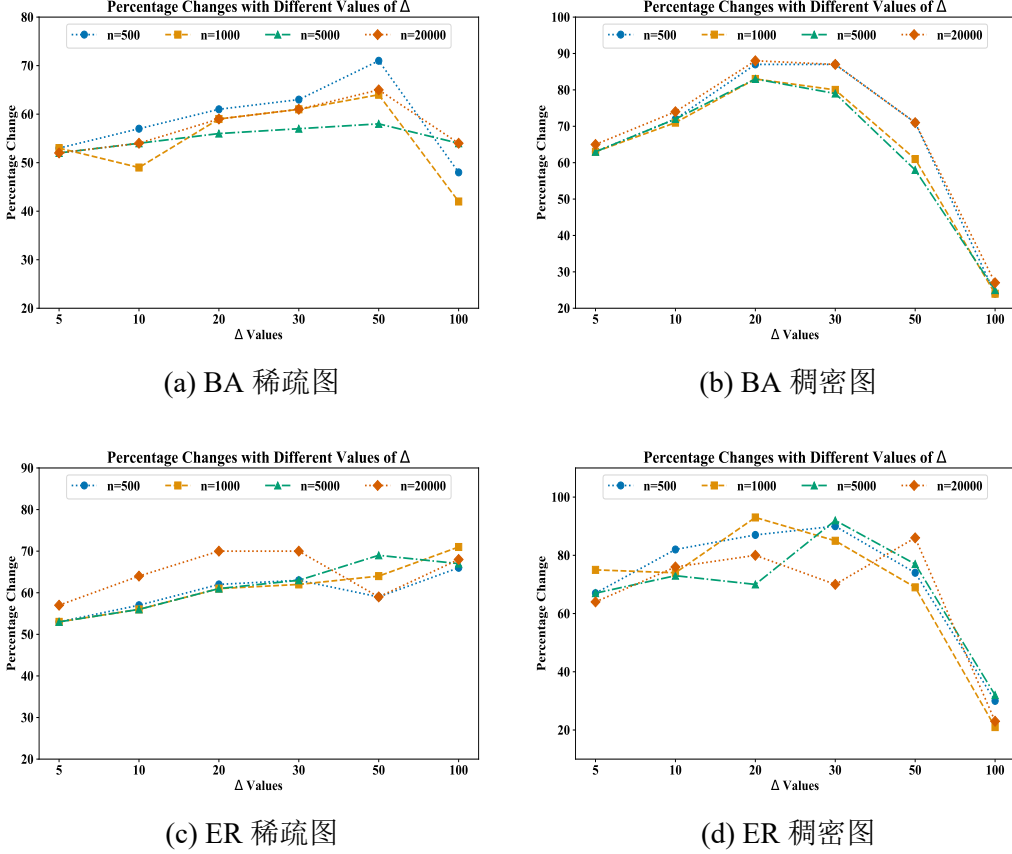


图 4-10 松弛算法在各网络图上的边松弛减少率

综合考虑加速率和边松弛减少率, 可以得出以下结论: 松弛算法在减少边松弛数量方面表现出色, 但同时也增加了边分类和判断无效边及叶顶点的条件计算负担. 因此, 松弛算法的表现效果取决于边松弛减少带来的收益与计算负担之间的平衡. 当边松弛减少的收益大于整体计算增加的负担时, 松弛算法表现出较好的效果, 加速率显著提高. 然而, 当边松弛减少的收益小于整体计算增加的负担时, 松弛算法的表现效果可能一般, 甚至可能出现负优化现象, 即加速率小于 0.

4.4 Δ 参数值的优化效果分析

前文分析已经明确指出, Δ 的取值直接影响 Δ -Stepping 算法的边松弛次数和桶的个数. 且合适的 Δ 取值能够显著减少算法的运行时间, 从而提高更新最短路径的效率. 因此, 选取一个合适的 Δ 值对于优化算法性能至关重要.

为验证第三章提出的最短路径树确定 Δ 的初始值的优化效果, 本节依次从常见网络图中选取了顶点数为 500, 1000, 5000, 10000, 20000 的稀疏图和稠密图进行实验. 并且设置 Δ 参数组为 5、10、20、30、50、100. 为更好地展示运行时间的差异, 对纵坐标间隔进行非等距处理. 图4-11和图4-12分别展示了在不同规模图下, 使用最短路径树确定的 Δ 值的运行时间, 其中红色实线代表实验结果. 为减少实验误差, 本节实验结果均基于五次重复实验计算得出平均值.

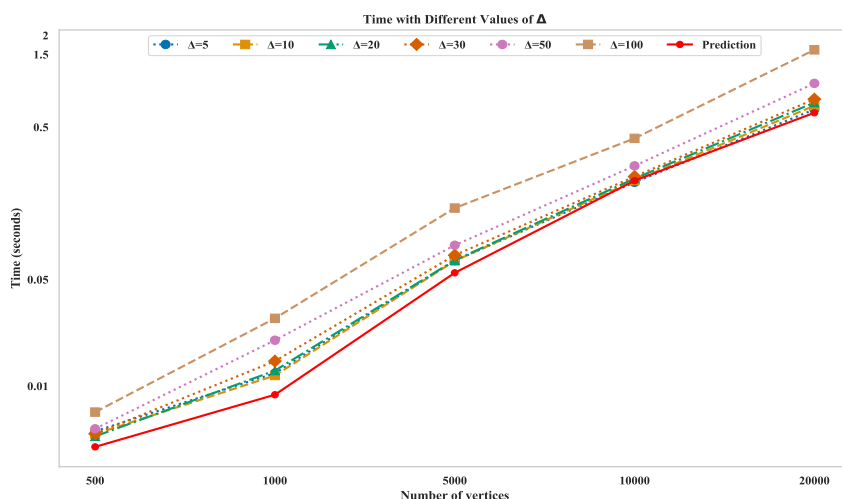


图 4-11 不同参数 Δ 在稀疏图下的运行时间对比

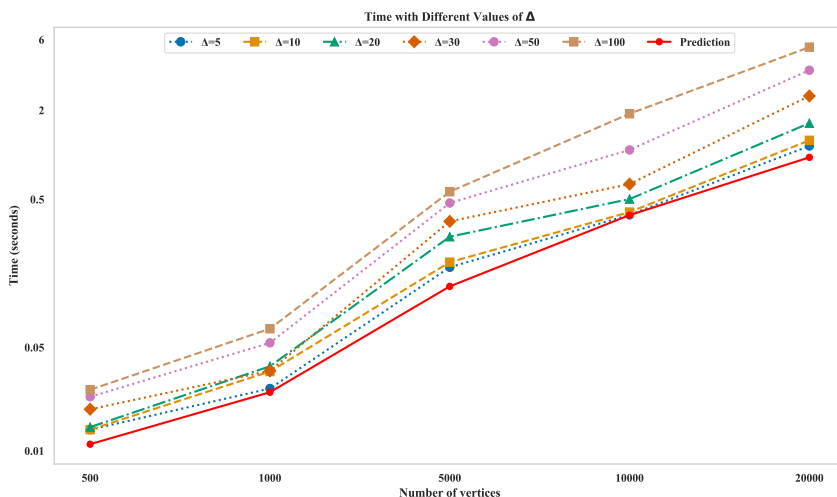


图 4-12 不同参数 Δ 在稠密图下的运行时间对比

根据图4-11和图4-12的展示, 可以看到, 预先通过最短路径树确定的 Δ 初始值在各个数据集上的表现效果普遍优于其他随机设定的 Δ 值. 这一结果充分验证了

最短路径树在确定 Δ 初始值方面的有效性. 同时, 注意到该算法在稠密图上的表现效果优于稀疏图, 这可以从稠密图中不同折线间较大的差距中直观看出. 然而, 由于最短路径树确定 Δ 初始值之后不会随着桶的顶点数和松弛边数进行动态调整, 这在一定程度上限制了其优化效果的进一步提升.

4.5 并行实验分析

前面已经对本文提出的优化算法在串行实验环境下进行验证. 接下来, 需要基于 GPU 并行实现进行实验分析, 本节选取顶点数为 5000 和 20000 的 BA 无标度网络的稀疏图和稠密图作为实验数据集. 并且为了研究 Δ 参数设置对实验效果的影响, 设置 Δ 参数组为 5、10、20、30 和 50, 并对比其优化效果. 此外, 为了观察最短路径树确定 Δ 初始值的并行算法效果, 会在表4-5中新增一列展示其并行化运行时间. 本节加速率的计算采取的是与 Δ -Stepping 原始算法并行运行时间进行对比, 所以加速率公式的 T_0 是 Δ -Stepping 原始算法并行实现时间. 为减少实验误差, 本节实验结果均基于五次重复实验计算得出平均值.

表 4-4 Δ -Stepping 原始算法串行与并行运行时间对比 (单位: 秒)

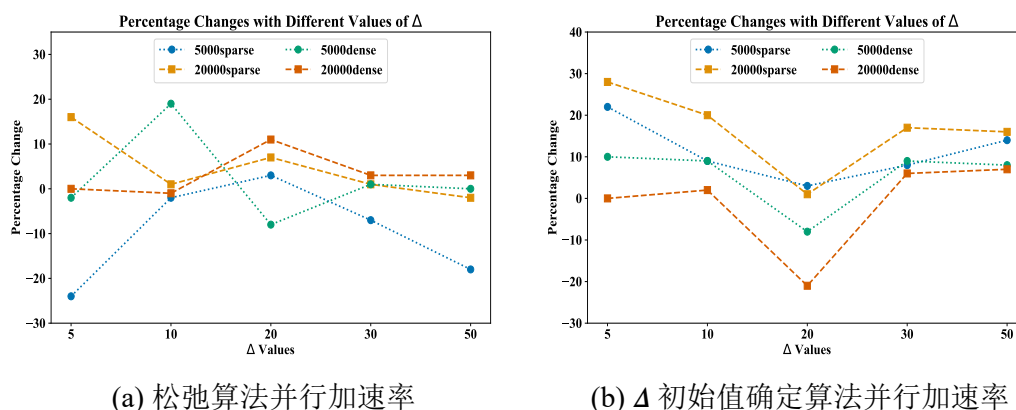
顶点数	性质	方式	$\Delta=5$	$\Delta=10$	$\Delta=20$	$\Delta=30$	$\Delta=50$
5000	稀疏	串行	0.0666	0.0664	0.0671	0.0724	0.0841
		并行	0.0112	0.0097	0.0090	0.0095	0.0102
5000	稠密	串行	0.1743	0.1886	0.2802	0.3559	0.4736
		并行	0.0206	0.0203	0.0171	0.0203	0.0200
20000	稀疏	串行	0.6544	0.6919	0.7264	0.7644	0.9708
		并行	0.0321	0.0290	0.0234	0.0280	0.0275
20000	稠密	串行	1.1500	1.2554	1.6394	2.5033	3.7359
		并行	0.0725	0.0738	0.0594	0.0764	0.0774

从表4-4中, 可以看到基于 GPU 并行实现的 Δ -Stepping 算法相较于串行实现具有显著的性能提升. 具体来说, 并行算法的平均时间开销仅为串行实现的 4.1%, 这充分证明了并行算法在优化效果上的优越性. 特别是在处理大规模稠密图时, 这种优化效果更为显著, 20000 顶点数的稠密图在并行实现下的时间开销仅为串行实现的 3.5%. 接下来将前文提到的松弛算法和最短路径树确定 Δ 初始值并行实现与原始 Δ -Stepping 并行实现进行对比分析.

表 4-5 Δ -Stepping 松弛优化算法并行运行时间 (单位: 秒)

顶点数	性质	$\Delta=5$	$\Delta=10$	$\Delta=20$	$\Delta=30$	$\Delta=50$	Δ 初始值确定
5000	稀疏	0.0139	0.0099	0.0087	0.0102	0.0121	0.0088
5000	稠密	0.0211	0.0165	0.0186	0.0200	0.0200	0.0185
20000	稀疏	0.0271	0.0286	0.0218	0.0278	0.0280	0.0231
20000	稠密	0.0724	0.0743	0.0529	0.0742	0.0752	0.0722

综合对比表4-4和表4-5, 松弛算法与最短路径树确定 Δ 初始值的并行效果均佳, 优化后运行时间均低于 0.1 秒. 并且图4-13显示, 两者并行加速率良好, 但相较于串行实现, 优化效果并不显著. 特别是松弛算法的并行化, 在 Δ 值过大或过小时, 原始算法的并行效果更佳. 此外, 值得注意的是, 与串行实现相比, Δ 值在并行环境下会对并行效率产生影响. 这种影响导致在利用最短路径树确定 Δ 初始值的并行实现过程中, 其加速率可能出现负值.

图 4-13 各优化算法并行在不同 Δ 下的加速率

4.6 真实图实验分析

前面 Δ -Stepping 算法的优化在随机生成图上实验的优化效果比较好, 为了增强实验说服力, 需要对现实生活中真实图数据进行实验对比分析. 真实图数据详细信息如下表4-6所示, 其中 wikipedia-links 和 livemocha 是稠密图, twitter 和 flickr-links 是稀疏图, 其中 n 表示图的顶点数, m 表示图中边的数量, $deg = \frac{m}{n}$ 表示图的平均度数. 由于各真实图只有顶点之间边的连接关系, 缺少边的权重, 所以对其随机赋 [1,100] 之间的权重, 并且为了研究 Δ 参数设置对实验效果的影响, 设置 Δ 参数组为 5、10、20、30 和 50 对比其优化效果. 关于本节加速率的计算, 串行实验取原

始算法串行运行时间为 T_0 ，并行实验取原始算法并行实现的运行时间为 T_0 。为减少实验误差，本节实验结果均基于五次重复实验计算得出平均值。

表 4-6 真实图数据集

数据集	n	m	deg
wikipedia-links	12949	637529	49.2338
twitter	23370	33101	1.4164
livemocha	104103	2193083	21.0664
flickr-links	1715255	15551250	9.0664

4.6.1 真实图串行实验分析

从表4-7和图4-14中的实验结果可以看出，在串行情况下，无论是松弛算法还是最短路径树确定 Δ 初始值的策略，在真实图数据上的优化效果都是比较显著的。这与网络生成图上的实验结果是一致的，进一步验证了这两种优化策略的有效性。对松弛算法而言，稠密图上的表现效果优于稀疏图，这是由于松弛算法对稠密图松弛次数减少带来的增益比较明显。相比之下，在稀疏图中，边的数量较少，松弛算法减少的松弛次数不足以弥补其增加条件判断计算所带来的开销，因此在某些情况下可能会出现负优化的情况，如 twitter 数据集。

表 4-7 各真实图原始算法串行运行时间对比 (单位：秒)

数据集	算法	$\Delta=5$	$\Delta=10$	$\Delta=20$	$\Delta=30$	$\Delta=50$	Δ 初始值确定
wikipedia-links	原算法	0.9096	0.9241	1.3273	1.4042	1.0258	0.9715
	松弛算法	0.5716	0.5363	0.5036	0.6109	0.7918	
twitter	原始算法	0.1002	0.1034	0.0985	0.0952	0.0946	0.0969
	松弛算法	0.0972	0.0994	0.0996	0.0998	0.0990	
livemocha	原始算法	29.623	26.772	26.999	27.935	37.386	21.574
	松弛算法	20.315	21.351	22.322	25.901	33.166	
flickr-links	原始算法	2295.1	2418.1	2605.2	2845.6	3399.8	2250.0
	松弛算法	2016.5	2270.7	2227.8	2482.2	2961.6	

从松弛算法和 Δ 初始值确定两种优化策略的对比来看，松弛算法在加速率方面确实表现出了优势，平均加速率高出 Δ 初始值确定策略 5%。与原始算法相比，松

弛算法最高加速率可以达到 62%，这充分说明了松弛算法在整体优化效果上的优越性。但是松弛算法在 Δ 较大时，松弛算法所带来的优化效果并不明显，在稠密图中这一现象更为明显，这是因为 Δ 较大时顶点往往都集中在其中几个桶，导致松弛次数的减少所带来的收益相对较低。

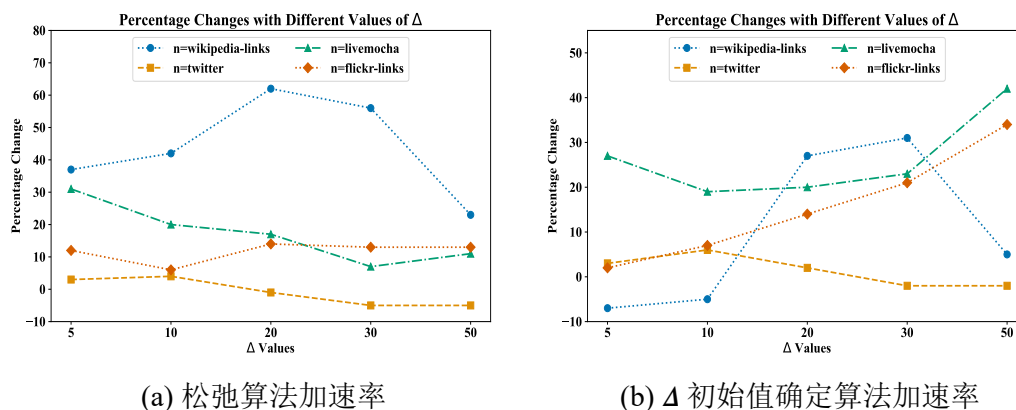


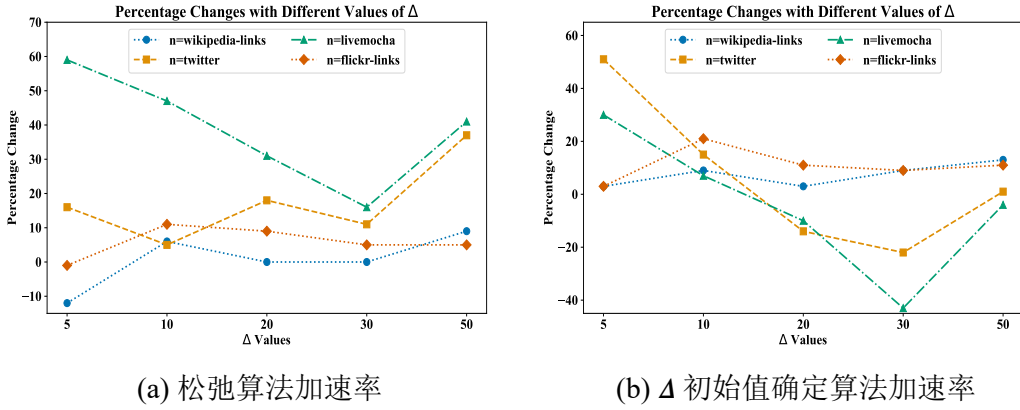
图 4-14 各优化算法串行在不同 Δ 下的加速率

4.6.2 真实图并行实验分析

从表4-8和图4-15可以发现，松弛算法加速率基本都大于 0。但是在并行方式下，松弛算法和最短路径树确定 Δ 初始值的优化效果并没有串行方式下明显。这是因为此时并行在算法运行时间缩短中起主要作用，因此松弛算法和最短路径树确定 Δ 初始值的优化效果会被弱化。

表 4-8 各真实图原始算法并行运行时间对比 (单位：秒)

数据集	算法	$\Delta=5$	$\Delta=10$	$\Delta=20$	$\Delta=30$	$\Delta=50$	Δ 初始值确定
wikipedia-links	原算法	0.0708	0.0750	0.0704	0.0750	0.0790	0.0684
	松弛算法	0.0792	0.0704	0.0702	0.0752	0.0722	
twitter	原算法	0.0175	0.0100	0.0075	0.0070	0.0086	0.0085
	松弛算法	0.0147	0.0095	0.0062	0.0062	0.0054	
livemocha	原算法	0.2050	0.1546	0.1304	0.1004	0.1383	0.1440
	松弛算法	0.0834	0.0826	0.0901	0.0843	0.0819	
flickr-links	原算法	0.3507	0.4308	0.3842	0.3729	0.3847	0.3409
	松弛算法	0.3552	0.3840	0.3498	0.3547	0.3647	

图 4-15 各优化算法并行在不同 Δ 下的加速率

同时也可以观察到, 对于大规模图 livemocha 和 flickr-links, 并行化效果非常明显, 其并行实现时间开销不到串行实现时间的 1%. 在适当的 Δ 值下, 松弛算法的运行时间和加速率都能达到较好的水平. 然而, 在并行化过程中, 特别是利用最短路径树确定 Δ 初始值的优化效果往往并不理想. 这主要是因为对于某些特定的图结构, 如 twitter 稀疏图 (顶点平均度数 $deg = 1.4164$), 其最短路径树的路径相对较长. 这种较长的路径长度在算法求解 Δ 初始值时产生了影响, 导致得到的 Δ 值偏小, 从而限制算法的并行化效率.

4.7 本章小结

本章首先介绍了实验所使用的实验环境、实验数据和评价指标. 接着对单源最短路径算法在不同规模图下的运行时间、边松弛次数和桶个数进行了比较分析. 然后从松弛算法优化, 最短路径树确定 Δ 值和并行优化三个方面分别与原始算法进行对比分析. 最后在真实图上对优化的 Δ -Stepping 算法与原始 Δ -Stepping 算法进行了串行实现和并行实现下的对比分析.

通过经典算法对比实验结果可以看出, Δ -Stepping 算法在边松弛次数和桶个数处于比较适中的水平, 而且在不同规模图中的运行时间表现也比较好. Δ -Stepping 优化实验结果表明, 通过减少边松弛次数, 松弛算法能够缩短算法的运行时间, 且在各规模稀疏图中平均加速率为 13%, 在各规模稠密图中平均加速率为 25%. 通过最短路径树确定 Δ 初始值, 是随机设定的一组 Δ 参数值中表现效果比较好的. 此外, 相比于串行实现, 基于 GPU 并行实现的优化效果非常显著, 网络图上的加速率基本都在 80% 以上. 而且以上实验结论在真实图上也得到了确认, 并且并行效果在大规模真实图上的表现尤为显著, 时间开销不到串行实现的 1%.

第五章 总结与展望

本章主要对前面的 Δ -Stepping 算法改进及其实验进行总结，并对后续研究给出展望。

5.1 总结

最短路径问题研究兼具理论意义和应用价值。本文针对单源最短路径问题，对 Dijkstra, Bellman-Ford 和 Δ -Stepping 三种经典算法进行理论分析和实验对比，结果发现：在同一规模图下 Δ -Stepping 算法运行时间相比 Dijkstra 算法和 Bellman-Ford 算法来说更少，且在顶点数比较高的稠密图上更为明显。从边松弛次数和桶个数角度分析， Δ -Stepping 算法边松弛次数和桶个数都处于 Dijkstra 算法和 Bellman-Ford 算法之间。总的来说， Δ -Stepping 算法不仅可以通过调节 Δ 参数值实现 Dijkstra 算法和 Bellman-Ford 算法的变体，结合了两种算法的优势。

因此本文基于 Δ -Stepping 算法做了三方面改进：(1) **边松弛次数优化**。通过轻重边的重新定义和剪枝两种优化策略对边松弛次数进行改善。对于重新定义轻重边的 EDS 算法，可以发现对于顶点数比较少时，重新定义带来的计算负担可能会对算法造成负优化，但是总体优化效果比较好，尤其是对于稠密图。对于剪枝优化 PDS 算法，在合适的 Δ 参数选择下，加速率可达 50% 以上，并且在各规模图下平均加速率为 28%。两种优化策略的叠加使用，平均边松弛减少率为 63%，且稠密图上的优化效果比较显著。(2) **Δ 参数值的优化**。通过最短路径树确定 Δ 的初始值，由实验结果分析可得知，此 Δ 的初始值的确定是不同规模图下运行时间比较少的。但是静态的 Δ 由于最短路径树容易受长距离和高深度的影响，所以仍然有一定优化空间。(3) **基于 GPU 并行化**。基于 GPU 采取清桶阶段顶点并行和松弛阶段边分配并行方式对 Δ -Stepping 算法进行优化。数值实验结果表明改进有效，而且在大规模图下并行实现优化效果更为显著。但是松弛算法和最短路径树确定 Δ 的初始值在并行实现的情况下，优化效果会受到一定影响。这是因为 Δ 值的确定在并行实现下会对算法的并行效率产生影响，当 Δ 值选取适当的时候，可以有效发挥并行优化的作用。因此在并行实现情况下，需要有效地平衡 Δ 值的选择，以确保并行效率的提升。

5.2 展望

虽然本文对 Δ -Stepping 算法改进取得一定成效，但是仍存在一些不足之处有待完善：

- (1) 本文主要针对解决静态图的单源最短路径求解，然而，在现实生活中，动态图上的单源最短路径问题同样具有重要意义。因此，后续研究将转向动态图的最短路径问题，以更好地应对实际场景中的挑战和需求。
- (2) 在本文中，主要探讨了使用最短路径树来确定 Δ 值的初始值，并以此作为优化算法性能的一种策略。然而，这种确定方式并没有考虑到并行计算过程中的线程利用率和顶点最短距离的收敛率，这可能导致在某些情况下负载不均衡，从而影响并行计算的效率。未来的研究将致力于将 Δ 值的确定与线程利用率和顶点收敛率相结合，以实现更精细的动态调整。
- (3) 尽管本文利用基于 GPU 的 CUDA 框架对 Δ -Stepping 算法进行了并行化，但在实现过程中，采用了固定的线程数和网格数量来配置核函数的块。这种做法虽然简单直接，但没有考虑到算法不同阶段处理任务数量的差异，可能导致线程利用率不足或某些阶段计算资源过剩的问题。在未来的研究中，将考虑引进动态并行策略，从而自适应调整任务分配策略，并提高线程利用率。

参考文献

- [1] Fu L, Sun D, Rilett L R. Heuristic shortest path algorithms for transportation applications: State of the art[J]. Computers & Operations Research, 2006, 33(11):3324-3343.
- [2] Bertsekas D. Dynamic behavior of shortest path routing algorithms for communication networks[J]. IEEE Transactions on Automatic Control, 1982, 27(1):60-74.
- [3] Tian X, Song Y, Wang X, et al. Shortest path based potential common friend recommendation in social networks[C]//2012 Second International Conference on Cloud and Green Computing. IEEE, 2012:541-548.
- [4] Arslan H, Manguoglu M. A parallel bio-inspired shortest path algorithm[J]. Computing, 2019, 101(8):969-988.
- [5] Dijkstra E W. A note on two problems in connexion with graphs[J]. 2022:287-290.
- [6] Fredman M L, Tarjan R E. Fibonacci heaps and their uses in improved network optimization algorithms[J]. Journal of the ACM (JACM), 1987, 34(3):596-615.
- [7] Fredman M L, Willard D E. Blasting through the information theoretic barrier with fusion trees[C]//STOC '90: Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing. New York, NY, USA: Association for Computing Machinery, 1990:1-7.
- [8] Ahuja R K, Mehlhorn K, Orlin J, et al. Faster algorithms for the shortest path problem[J]. J. ACM, 1990, 37(2):213-223.
- [9] Deng Y, Chen Y, Zhang Y, et al. Fuzzy dijkstra algorithm for shortest path problem under uncertain environment[J]. Applied Soft Computing, 2012, 12(3):1231-1237.
- [10] Pinandito A, Kharisma A P, Akbar M A. Improving route-finding performance of dijkstra algorithm and maintaining path visual cue using douglas-peucker algorithm[C]//Proceedings of the 8th International Conference on Sustainable Information Engineering and Technology. 2023:401-408.
- [11] Bellman R. Dover books on computer science: Dynamic programming[M]. Dover Publications, 2013.
- [12] Bellman R. On a routing problem[J]. Quarterly of Applied Mathematics, 1958, 16:87-90.
- [13] Yen J Y. An algorithm for finding shortest routes from all source nodes to a given destination in general networks[J]. Quarterly of applied mathematics, 1970, 27(4):526-530.
- [14] Bannister M J, Eppstein D. Randomized speedup of the bellman-ford algorithm[C]//2012 Proceedings of the Ninth Workshop on Analytic Algorithmics and Combinatorics (ANALCO). SIAM, 2012:41-47.
- [15] 段凡丁. 关于最短路径的 SPFA 快速算法 [J]. 西南交通大学学报, 1994, 29(2):6.
- [16] Parimala M, Broumi S, Prakash K, et al. Bellman-ford algorithm for solving shortest path problem of a network under picture fuzzy environment[J]. Complex & Intelligent Systems, 2021, 7:2373-2381.

-
- [17] Meyer U, Sanders P. δ -stepping: a parallelizable shortest path algorithm[J]. *Journal of Algorithms*, 2003, 49(1):114-152.
 - [18] Blleloch G E, Gu Y, Sun Y, et al. Parallel shortest paths using radius stepping[C]//*Proceedings of the 28th ACM Symposium on Parallelism in Algorithms and Architectures*. 2016:443-454.
 - [19] Dhulipala L, Blleloch G, Shun J. Julienne: A framework for parallel graph algorithms using work-efficient bucketing[C]//*Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures*. 2017:293-304.
 - [20] Dong X, Gu Y, Sun Y, et al. Efficient stepping algorithms and implementations for parallel shortest paths[C]//*Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures*. 2021:184-197.
 - [21] Lei G, Dou Y, Li R, et al. An fpga implementation for solving the large single-source-shortest-path problem[J]. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2015, 63(5): 473-477.
 - [22] Busato F, Bombieri N. An efficient implementation of the bellman-ford algorithm for kepler gpu architectures[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2015, 27(8):2222-2233.
 - [23] Chakaravarthy V T, Checconi F, Murali P, et al. Scalable single source shortest path algorithms for massively parallel systems[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2016, 28(7):2031-2045.
 - [24] Yu H, Wang X, Luo Y. An edge-fencing strategy for optimizing sssp computations on large-scale graphs[C]//*ICPP '21: Proceedings of the 50th International Conference on Parallel Processing*. New York, NY, USA: Association for Computing Machinery, 2021.
 - [25] Ding W, Lin G. Partially dynamic single-source shortest paths on digraphs with positive weights[C]//*International Conference on Algorithmic Applications in Management*. Springer, 2014:197-207.
 - [26] Riazi S, Srinivasan S, Das S K, et al. Single-source shortest path tree for big dynamic graphs[C]//*2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2018:4054-4062.
 - [27] Jabbar L S, Abass E I, Hasan S D. A modification of shortest path algorithm according to adjustable weights based on dijkstra algorithm[J]. *Engineering and Technology Journal*, 2023, 41(2):359-374.
 - [28] Floyd R W. Algorithm 97: shortest path[J]. *Communications of the ACM*, 1962, 5(6):345.
 - [29] Johnson D B. Efficient algorithms for shortest paths in sparse networks[J]. *Journal of the ACM (JACM)*, 1977, 24(1):1-13.
 - [30] Han Y, Pan V, Reif J. Efficient parallel algorithms for computing all pair shortest paths in directed graphs[C]//*Proceedings of the fourth annual ACM symposium on Parallel algorithms and architectures*. 1992:353-362.
 - [31] Takaoka T. A new upper bound on the complexity of the all pairs shortest path problem[J]. *Information Processing Letters*, 1992, 43(4):195-199.
 - [32] Han Y, Takaoka T. An $O(n^3 \log \log n / \log^2 n)$ time algorithm for all pairs shortest paths[J]. *Journal of Discrete Algorithms*, 2016, 38:9-19.

-
- [33] Bondhugula U, Devulapalli A, Fernando J, et al. Parallel fpga-based all-pairs shortest-paths in a directed graph[C]//Proceedings 20th IEEE International Parallel & Distributed Processing Symposium. IEEE, 2006:10-pp.
- [34] Katz G J, Kider J T. All-pairs shortest-paths for large graphs on the gpu[J]. 2008.
- [35] Schoeneman F, Zola J. Solving all-pairs shortest-paths problem in large graphs using apache spark[C]//Proceedings of the 48th International Conference on Parallel Processing. 2019:1-10.
- [36] Yang S, Liu X, Wang Y, et al. Fast all-pairs shortest paths algorithm in large sparse graph[C]//Proceedings of the 37th International Conference on Supercomputing. 2023:277-288.
- [37] Hanif M K, Zimmermann K H, Anees A. Accelerating all-pairs shortest path algorithms for bipartite graphs on graphics processing units[J]. Multimedia Tools and Applications, 2022, 81 (7):9549-9566.
- [38] Sao P, Lu H, Kannan R, et al. Scalable all-pairs shortest paths for huge graphs on multi-gpu clusters[C]//Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing. 2021:121-131.
- [39] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE transactions on Systems Science and Cybernetics, 1968, 4(2):100-107.
- [40] Chabini I, Lan S. Adaptations of the a* algorithm for the computation of fastest paths in deterministic discrete-time dynamic networks[J]. IEEE Transactions on intelligent transportation systems, 2002, 3(1):60-74.
- [41] Ju C, Luo Q, Yan X. Path planning using an improved a-star algorithm[C]//2020 11th International Conference on Prognostics and System Health Management (PHM-2020 Jinan). IEEE, 2020:23-26.
- [42] Dorigo M, Maniezzo V, Colorni A. Ant system: optimization by a colony of cooperating agents[J]. IEEE transactions on systems, man, and cybernetics, part b (cybernetics), 1996, 26(1):29-41.
- [43] 易正俊, 李勇霞, 易校石. 自适应蚁群算法求解最短路径和 TSP 问题 [J]. 计算机技术与发展, 2016, 26(12):1-5.
- [44] Jingwei Z, Ting R, Ming L, et al. Simulated annealing ant colony algorithm based on backfire method for qap[C]//Proceedings of the 4th International Conference on Internet Multimedia Computing and Service. 2012:100-105.
- [45] Chen S, Sun Y, Song H, et al. An improved ant colony algorithm based on target-driven gravity force[C]//ICTCE '22: Proceedings of the 2022 5th International Conference on Telecommunications and Communication Engineering. New York, NY, USA: Association for Computing Machinery, 2023:281-288.
- [46] Gordon V S, Whitley D. Serial and parallel genetic algorithms as function optimizers[C]//ICGA. 1993:177-183.
- [47] 赵辉, 徐俊刚. 基于 OpenMP 多核架构下并行蚁群算法研究 [J]. 微型机与应用, 2011, 30 (16):6-8.
- [48] Akiba T, Iwata Y, Yoshida Y. Fast exact shortest-path distance queries on large networks by pruned landmark labeling[C]//Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data. 2013:349-360.

- [49] Saad Y, Schultz M H. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems[J]. SIAM Journal on scientific and statistical computing, 1986, 7(3): 856-869.
- [50] Erdős P, Rényi A. On random graphs i[J]. Publ. math. debrecen, 1959, 6(290-297):18.
- [51] Barabási A L, Albert R. Emergence of scaling in random networks[J]. science, 1999, 286(5439):509-512.
- [52] 张钟. 大规模图上的最短路径问题研究 [D]. 中国科学技术大学, 2014.
- [53] 陈钧吾, 余华山. 面向无尺度图的 Δ -stepping 算法改进策略 [J]. 计算机科学, 2022.
- [54] Kunegis J. KONECT – The Koblenz Network Collection[C]//Proc. Int. Conf. on World Wide Web Companion. 2013:1343-1350.

致 谢

行文至此，意味着二十余载求学生涯即将结束。回首过去，总感觉自己非常幸运，虽然途中不乏曲折与坎坷，但每一次的历练都化作了内心的喜悦与成就。我深信，这些幸运很多都来自于身边的人，正是他们在我迷茫时，在我骄傲时，在我失意时，不断地鼓励，包容与安慰我，让我能够坚定地走到今天。

首先，我要向我的导师胡平老师和姚正安老师致以最深的谢意。在毕业论文的撰写过程中，从选题的初步构思到中期的汇报，再到最终的论文定稿，两位老师耐心地解答我的疑惑，细致地指导我修正错误，严格地要求我精益求精。正是有了他们的指引，我才能够克服种种困难，最终完成这篇论文。

同时，我也要感谢课题组的 Henry 老师和张赞波老师，他们每次以耐心和包容的态度倾听我汇报论文，并给出我汇报论文时存在的不足以及未来可以研究的方向。也正是因为他们的宝贵的建议和指导，我才能逐步积累知识，不断进步。

其次，我还要感谢课题组的张奇师兄、唐荣霞师姐和幸俊龙师兄。他们在我的学习和生活中都给予了极大的帮助和支持。此外，我还要感谢课题组的黄志强、罗宗培、姚嘉琦和曾嘉馨等同学，我们一同学习、共同进步，记得我们一起在南门聚餐的瞬间，一起过生日的场景，那段在测中五楼的时光将是我宝贵的回忆。我还要特别感谢我的室友刘珍。她总是能够发现身边每个人的闪光点，鼓励我们勇敢地尝试新的事物，她的积极态度给了我无尽的动力。同时，我要感谢我的朋友饶夏晴和倪雨。我们曾一同在图书馆二楼备战考研，也曾一起在蛟湖旁互相畅想梦想，规划未来。衷心祝愿他们未来能够在各自的领域里大放异彩。

再次，我要感谢我的母亲。虽然她的学历不高，但她用尽全力托举我，一直在为我创造更好的学习环境。始终无条件地支持我的每一个决定，让我在人生的重大抉择中能够坚定前行。希望未来我可以成为母亲的依靠，托举母亲走向广阔世界。

最后，感谢各位答辩专家在繁忙的工作中抽出时间参与我的论文评阅、评议和答辩。你们的宝贵意见和建议对我而言意义重大，我将珍视并付诸实践。